Java is a platform independent because it compiled into byte code which runs on the jvm. Jvm is available for different os , same bytecode cn run anywhere.

.java file --(javac compiler)--> .class file (bytecode)

.class file --(JVM)--> Machine Code

java source code is compiled by the javac compiler into bytecode (.class file). This bytecode is platform independent. The JVM, which is platform dependent, interprets or JIT-compiles this bytecode into machine code. Since JVM is available for multiple operating systems, the same bytecode can run anywhere. That's why Java is platform independent

The full line:

public static void main(String[] args)

- **1. public:** Means the method is **accessible from anywhere**.

👉 JVM (Java Virtual Machine) must be able to call it from outside your class.

- **2. static:** Means the method belongs to the **class itself**, not to any object.

👉 JVM can call it **without creating an object** of the class.

- **3. void:** Means the method **does not return anything**.

👉 It just executes your program — there's no output returned to JVM.

- **4. main:** This is the **method name**.

👉 JVM looks for this specific method name to start your program.

- **5. (String[] args):** This is a **parameter** — an array of String values.

👉 It stores **command-line arguments**, if you run your program like:

java Main Hello World

Then args[0] = "Hello" and args[1] = "World".

Use an **abstract class** when you want to share common code among related classes, and use an **interface** when you want to define a contract that multiple classes can implement in different ways.

🧠 **In simple words:**

“public static void main(String[] args) means:
a method that can be called by JVM without creating an object,
runs first, doesn’t return anything, and can take input from the command line.”

JVM coverts byte code to machine code so machine can understand, it does the memory management, it perform garbage collect, ensure platform independency

Compile: check no bugs,no error and make code non human readable

JRE: java run time environment to run java program it is required

JDK: java development kit which software development environment which is required for build java application

Garbage Collector

☑ Stack

- Stores **primitive values** and **references to objects**.
- Memory is allocated when a **method runs**.
- Follows **LIFO (Last In, First Out)**.
- This is thread specific

Example:

```
int a = 10;     // 'a' (value 10) is stored in the stack
String s = "Hi"; // reference 's' is in stack, actual "Hi" object is in Heap
```

☑ Heap

- Stores **all objects** created using new.
- Memory is allocated when JVM executes the line that creates the object.
- Shared among the threads.

Example:

```
Person p = new Person();
```

- The p reference is stored in the **stack**.
- The actual Person object is stored in the **heap**.

☑ Garbage Collection

- If an object in the heap **does not have any reference in the stack**, the **Garbage Collector deletes it**.

Example:

```
Person p = new Person();
p = null;   // now the Person object has NO reference
```

✓ The original Person object is eligible for **garbage collection**.

☑ Generational Garbage Collection

The heap is divided into parts:

1. **Young Generation** – where new objects are created
2. **Old Generation** – objects that survived for a long time
3. **Permanent Generation / Metaspace** – stores class definitions, metadata

4. **Example:**

- When you create many small objects in a loop, they first go into the **Young Gen**.
- If they stay alive longer, JVM moves them to the **Old Gen**.

Memory leak: when an obj is not getting used butstill it refferenced so can no remove it that is called leak.

Stop-the –world=during gc when all application thread are paused that state is called stop the world.

Mark: gc collector scan the heap memory to find the obj that are still reachable by the application.

Sweep: it removes the obj which are not reachable.

Compact: it rearrages the obj, it makes future allocation faster.

Different GC collector(G1,ZGC).

How to achieve concurrency?

Using multiple threads, executer service, synchrization

How many ways to define string:

1.Creating object of string,

2.By string literal

String is a bulitin class in java

Collection of char

Creating string list

```
 //creating string list
 String str[]={"kuhu","naj","paro"};
 String str2[]=new String[]{"kuhu","naj","paro","nasu"};
```

```
public static void main(String[] args) {
    String str[]={"kuhu","naj"};
    String str2[]=new String[]{"ami","tumi"};
    int arr[]={5,3,2,1};
    for (String x: str){
        System.out.println(x);
    }
    for (String x: str2){
        System.out.println(x);
    }
    for(int i:arr){
        System.out.println(i);
    }
```

Conversion of char of string into a single string

```
char arr[]={'A','b','v','d'};
String str=new String(arr);
System.out.println(arr);
```

Creating string

If we use obj then every time it will occupy a space in heap

And if we just assign a value to string like str3 then it will be act as a reference ,it will store in string pool

```
//different ways to create string
//creating a obj and there storing the value of string in heap
String str2=new String("kuhu");,
//creating a reference of string in pool
String str3="nasu";
//no reference will be created it will point to str3 only as both
    of them are storing same value
String str4="nasu";
System.out.println(str2);
System.out.println(str3);
```

Ranging(it will print bvd)

```
char arr[]={'A','b','v','d','n'};
//ranging
String str=new String(arr,1,3);
System.out.println(str);
```

== to use whether two variable reference to a same obj or not

```
String str1="java";
String str2="java";
String str3=new String("java");
//output true
System.out.println(str1==str2);
//output false
System.out.println(str1==str3);
```

Methods/function(static ,non static)

Our main function is static so calling a method that also be a static method

static int max(int a,int b)

if we are not using static then need to create obj then call the non static method

```
public class NonStaticMethod {
    int max(int a,int b) {
        if(a>b) {
            return a;
        }
        else {
            return b;
        }
    }

    public static void main(String[] args) {
        //creating obj then only nonstatic method we can call in static method
        NonStaticMethod n= new NonStaticMethod();
        int a=2;
        int b=9;
        int result=n.max(a, b);
        System.out.println("greater: "+result);
```

Methods in string

```
String str1="java program";
String str2="java program";
int size=str1.length();
String s=str1.toLowerCase();
String s2=str1.toUpperCase();
//till 6 it will print
String sub=str1.substring(3,7);
String replace=str1.replace('j','p');
char c=str1.charAt(6);
int s3=str1.indexOf("v");
System.out.println(s3);
System.out.println(str1.equals(str2));
```

```
String str1="java";
String str2="program";
System.out.println(str1.concat(str2));
 System.out.println(str1+ str2);
```

```
public static void main(String[] args) {
  String mail="nasima@gmail.com";
  int i=mail.indexOf("@");
  String username=mail.substring(0,i);
  String domainname=mail.substring(i+1,mail.length());
  boolean result=domainname.startsWith("gmail");
  System.out.println(result);
  System.out.println(domainname);
```

Display the digits and count the digits

```java
import java.util.*;
class HelloWorld {
    public static void main(String[] args) {
     int n=257;
     int count=0;
     while(n!=0){
        int remainder=n%10;
        System.out.println(remainder);
        n=n/10;
        count=count+1;

     }
     System.out.println(count);
     }
}
```

Armstrong or not

```java
 int n=163;
 int temp=n;
 int result=0;
while(n!=0){
    int remainder=n%10;
    result=result+remainder*remainder*remainder;
    n=n/10;

}
System.out.println(result);
if(result==temp){
     System.out.println("armstrong");

}
else{
    System.out.println("not");
}
```

Fibonacci series

```java
int n1=0,n2=1,n3,n=10;
System.out.println(n1+","+n2);
for(int i=2;i<n;i++){
    n3=n1+n2;
    System.out.println(n3);
    n1=n2;
    n2=n3;
}
```

Array

In java every is obj. every arr is created in heap

Declaration of arr:

```
// int arr[]=new int[5];
 int arr[]={3,2,4,1};
 for(int i=arr.length-1;i>=0;i--){
     System.out.println(arr[i]);
 }
 //for each method
```

For each in java

```
//for each method
for(int x:arr){
    System.out.println(x);
}
String cars[]={"volvo","a","b"}
for (String x:cars){
    System.out.println(x);
}
```

Search element

```
 int arr[]={3,2,4,1};
 int sum=0;
 int elem=9;
 int found=0;
 for(int i=0;i<arr.length;i++){
     if(arr[i]==elem){
         found=1;
         break;
     }
 }
 if(found>0){
     System.out.println("present");
 }
 else{
     System.out.println("not present");
 }
```

```
int elem=8;
boolean found=false;
for(int i=0;i<arr.length;i++){
    if(arr[i]==elem){
        found=true;
       System.out.println("found at"+arr[i]);

    }

}
if(!found){
    System.out.println("found not");
}
```

```
    for(int i=0;i<arr.length;i++){
        if(arr[i]==elem){
           System.out.println("found at"+arr[i]);
           System.exit(0);

        }

    }

   System.out.println("found not");
```

Largest and second largest

```
 int arr[]={3,2,4,9};
int max=arr[0];
int sec=arr[0];
 boolean found=false;
 for(int i=0;i<arr.length;i++){
     if(arr[i]>max){
         sec=max;
         max=arr[i];
     }
     else if(arr[i]>sec &&arr[i]!=max){
         sec=arr[i];
     }
 }
```

Add new elem in last

```java
int arr[]={3,2,4,9};
int x=8;
int size=arr.length;
int newArr[]=new int[size+1];
for(int i=0;i<size;i++){
    newArr[i]=arr[i];
}
newArr[size]=x;
for(int i:newArr){
    System.out.println(i);
}
```

Data types:

The main difference between **primitive** and **non-primitive** data types are:

Primitive types are predefined (already defined) in Java. Non-primitive types are created by the programmer and is not defined by Java (except for `String`).

Non-primitive types can be used to call methods to perform certain operations, while primitive types cannot.ex: Scanner

A primitive type has always a value, while non-primitive types can be `null`.

A primitive type starts with a lowercase letter, while non-primitive types starts with an uppercase letter.

Primitive values stored in stack and non in heap

Size of int =4 bytes

Strings are immutable

Bit mask: manipulating individual bits of number using bitwise operator(AND,OR)

Wrapper class:

It converts primitive type into objects so it can be used in collection

int a=10;

Integer b=a;

Local var :are stayed in stack declared inside a method,

class var :are shared among all objects, defined as static it shared by among all the obj , it is created when a class is loadded.it not belongs to a

particular obj. Static members are created **when class loads**, even before object creation.

Instance var :stayed in heap it belongs to objects.it is created when a obj is created.

❌ When NOT to Use Static

If data is **unique per object**, do not use static.

```
class Demo {

    int x = 10;
    static int y = 20;

    static void show() {
        // System.out.println(x); ❌ Error
        System.out.println(y);  // ☑ Works
    }
}
```

👉 "Can static method access instance variable?"

Answer:

"No, because static method belongs to class and instance variable belongs to object. Without object reference, it cannot access instance data."

```
class Main {
    //a var which is declared a static inside a class,
    //it shared by among all objects.
    static int a=10;
    //instance var
    int b=9;
```

this. -> Will tell the function which object is calling /

It will remove the conflict between argument and instance variable

//example of this keyword

```
class thisExample{
    //class variable
    int lenth,bredth;
    //parameterized construcor
    thisExample(int l,int b) {
        //assiging value to the class variable using instance using this
        this.lenth=l;
        this.bredth=b;

    }
    public void display() {
        //we can this. or just the variable name
        System.out.println("Length:"+this.lenth);
        System.out.println("bred:"+bredth);
    }
}
public class ThisAndFinal {

    public static void main(String[] args) {
        thisExample t=new thisExample(20,30);
        thisExample t1=new thisExample(70,80);
t.display();
```

constructor->creating new objects

no return types

3 types of construct 1.parameterize,no para,copy.

Product of 2 number

Destructor: deleting objects which are no use

Variable arguments(as much we can pass the value using …)

```java
import java.util.Scanner;
class HelloWorld {
    static void StringList(String ...A){
        for(String x:A){
            System.out.println(x);
        }
    }
    static void arrayList(int  ...A){
        for(int  x:A){
            System.out.println(x);
        }
    }
    static void array(int arr[]){
        for(int x:arr){
            System.out.println(x);
        }
    }
    public static void main(String[] args) {
      //creating string list
      String str[]={"kuhu","naj","paro"};
      int arr[]={2,3,1};
      StringList(str);
      arrayList(arr);
```

What is OutOfMemoryError?

Simple Interview Answer:

"OutOfMemoryError occurs when JVM cannot allocate memory for new objects because the heap or other memory areas are full."

It is an **Error**, not Exception.

OOPS concept

Class: it is a blueprint or design of an obj as a example design of a toy

Object: it is a real thing made using that blueprint .

Reference of animal class

(parent class) ->Animal dog=new Dog()->obj of dog

Abstraction: hides the implmentation details/

Encapsulation/data hiding: it hides some properties it will deside to which properties to show which not . It wraps variable and methods together.it provide security layer

Polymorphism:helps to create a task in different ways.Names are same but behaviour is different like adding 2 int ,2 float,concat 2 string

2 types(run time,compile time)

Method overloading (compile time):in single class create same name class but parametes .

🔥 Why Runtime Polymorphism Works?

Because of:

👉 Dynamic Method Dispatch

At runtime JVM decides which method to call based on **object**, not reference type.

Example:

Animal a = new Dog();

Reference → Animal
Object → Dog

Method of Dog runs.

If we make the method static then when we are doing this parennt p=new Child()

Parent method gonna callled beacuse static method are belongs to class they are not belongs object.

But if dont assign static imean just int print() then in this case child method gonna called .

Dynamic dispatch/Method overriding(runtime):

Method Overloading	Method Overriding
Method overloading is a compile-time polymorphism.	Method overriding is a run-time polymorphism.
Method overloading helps to increase the readability of the program.	Method overriding is used to grant the specific implementation of the method which is already provided by its parent class or superclass.
It occurs within the class.	It is performed in two classes with inheritance relationships.
Method overloading may or may not require inheritance.	Method overriding always needs inheritance.
In method overloading, methods must have the same name and different signatures.	In method overriding, methods must have the same name and same signature.
In method overloading, the return type can or can not be the same, but we just have to change the parameter.	In method overriding, the return type must be the same or co-variant.
Static binding is being used for overloaded methods.	Dynamic binding is being used for overriding methods.
Private and final methods can be overloaded.	Private and final methods can’t be overridden.
The argument list should be different while doing method overloading.	The argument list should be the same in

Inheritance

it is possible to inherit attributes and methods from one class to another

single,hybrid,multilevel,hierarcial

access modifiers:

Default – No keyword required, accessible **only within the same package**.

private: only inside a class we can access

public : we can access it from any package

protected: **accessible within the same package or subclasses in different packages.**

Access private class we use getter to get value setter to set value;

Data hiding:using getter and setter method we are doing it we are making the variable private, to access them we are using getter and setter.

Constructor: it will have the same name as class name ,it will called automatically when a obj is created, 3 types: with parameter(passes argument with to set specific value while obj creation) and without parameter(passes no argumnets),copy constructor(use to copy obj form one to another)

The `toString()` method is automatically invoked when you try to print an object reference in Java. This happens because `System.out.println()` internally calls the `toString()` method of the object passed to it.

```
public Subject(String id,int marks) {
    subId=id;
    maxMark=marks;
}
public String toString() {
    return subId + maxMark;
}
```

Interface vs Abstraction

Abstract class	Interface

1) Abstract class can have abstract and non-abstract methods.	Interface can have only abstract methods. Since Java 8, it can have default and static methods also.
2) Abstract class doesn't support multiple inheritance.	Interface supports multiple inheritance.
3) Abstract class can have final, non-final, static and non-static variables.	Interface has only static and final variables.
4) Abstract class can provide the implementation of interface.	Interface can't provide the implementation of abstract class.
5) The abstract keyword is used to declare abstract class.	The interface keyword is used to declare interface.
6) An abstract class can extend another Java class and implement multiple Java interfaces.	An interface can extend another Java interface only.
7) An abstract class can be extended using keyword "extends".	An interface can be implemented using keyword "implements".
8) A Java abstract class can have class members like private, protected, etc.	Members of a Java interface are public by default.

9)Example:
public abstract class Shape{
public abstract void draw();
}
	Example:
	public interface Drawable{
	void draw();
	}

2 types of class 1.Abstract ((obj can not be created)2.concreate(obj can be created)

Abstraction:

showing only important details, hiding the implementation details.

cannot create obj of Abstruction class, it is designed o be subclasedd

can create constructor of abs class.it usefull for subclass intialization

It can have method with body and without body also

```
//can to be final
//this abs classes are ment for interitance
//if a method is abs then the class also should be abs.
//abs method can not be static method
abstract class Testing {
    public abstract void abs();

    public void show() {
        System.out.println("say hi to parent");
    }
}
```

```
abstract class New{
    public New() {
        System.out.println("constructor");
    }
    public void method1() {
        System.out.println("i am method 1");
    }
    abstract void method2();
}
//concreate class
class Sub extends New{
    public void method2(){
        System.out.println("i am method 2");
    }
}
```

Interface:

can not have constructor of interface ,

can not create non abstract class, and can do implementation,

by default methods are public , variable are static means non changables ;

multiple inheritance is possible here.can not create object of a interface

access specifier should be public

```java
interface methods {
    // override method
    public void printing();
}

//extending the interface
class inter implements methods {
    // Overridden method
    @Override
    public void printing() {
        System.out.println("say hi to overide interface");

    }

public static void main(String[] args) {
    //creating obj of class but reference it through the methods interface
    //it call call only those methods which are overriden in class
    methods t = new inter();
    inter t2 = new inter();
    //
    t.printing();
```

```java
//multilevel inheritance
class grandchild extends child {
    public grandchild() {
        System.out.println("i am grandchild");
    }

    public grandchild(int x) {
        // calling parameterized cont from parent clas
        super(x);
        System.out.println("i am param");
    }

}

public class Inheritance {

    public static void main(String[] args) {
//      B b=new B();
//      b.l=9;
//      b.bre=8;
//      System.out.println(b.volume());
        grandchild c = new grandchild(20);

    }
```

Super: Using super we can access the item of parent class,it is useful in interitance only to accessing the parent constructor.

```java
class thisExample{
    //class variable
    int lenth,bredth;
    int x=70;
    //parameterized construcor
    thisExample(int l,int b) {
        //assiging value to the class variable using instance using this
        this.lenth=l;
        this.bredth=b;

    }
    public void display() {
        //we can this. or just the variable name
        System.out.println("Length:"+this.lenth);
        System.out.println("bred:"+bredth);
    }
}
class example extends thisExample{
    int height;
    int x=90;
    example(int l,int b,int h){
        //accessing the len and bre of parent class thisExample
        super(l,b);
        this.height=h;
    }

    public void display() {
        //we can this. or just the variable name
        // Displaying parent and child class variables
        System.out.println("Length: " + super.lenth); // Access parent class variables
        System.out.println("Breadth: " + super.bredth);
        System.out.println("Height: " + height);
        System.out.println("Parent x: " + super.x); // Access parent class 'x'
        System.out.println("Child x: " + this.x);  // Access child class 'x'
    }
}
```

Why And When To Use Interfaces?

1) To achieve security - hide certain details and only show the important details of an object (interface).

2) Java does not support "multiple inheritance" (a class can only inherit from one superclass). However, it can be achieved with interfaces, because the class can **implement** multiple interfaces.

```java
// Interface
interface Animal {
  public void animalSound(); // interface method (does not have a body)
  public void sleep(); // interface method (does not have a body)
```

```
}
// Pig "implements" the Animal interface(accessing interface
methods)
class Pig implements Animal
```

inner class vs static inner class

Nested classes are divided into two categories: static and non-static. Nested classes that are declared static are simply called static nested classes. Non-static nested classes are called inner classes.

Static: when we have common info as like school name in that case we make that var as static. static variable will be shared by ever obj ,static var belongs to class .

Static variables and methods in Java provide several advantages, including memory efficiency, global access, object independence, performance, and code organization.

If any object is changing the value of static var it will change for every obj present in class.

Without creating obj also we can access them using classname.

Static methods can accees only static number .it can not access non static members.

ex: if I change the school name it will change for everyone.

Static String schoolName;

And no need to create obj to access static value,method directly can access with(className. schoolName)

If I am making any method static we can directly access them or by classname. also , no need to create obj/interface.if it is not static then need to access first create obj then call;

```java
public class Practice {
    public void addNum(int a,int b){
        System.out.println("add: "+ a+b);
    }
    public static void minusNum(int a,int b){
        System.out.println("miuns: "+ (a-b));
    }
    public static void main(String[] args) {
        int a=9;
        int b=8;
        Practice pc=new Practice();
        pc.addNum(a,b);
        Practice.minusNum(a,b);
        minusNum(a,b);
    }
}
```

Final Method:

Final variables can not be modifies

Final method can not be overriten

Final class cannot be inherited;

Ex: if I don't want some class to have ineritance, some method to overwritten ,some value to unchanged in those case cases final is workable.

Simple One-Line Difference

- **final = reference cannot be changed.**
- **Immutable = object's content cannot be changed.**

Access /modifiers

outer class can be only default or public

Default: The data members, classes, or methods that are not declared using any access modifiers i.e. having default access modifiers are accessible **only within the same package**.

Private: The methods or data members declared as private are accessible only **within the class** in which they are declared.

Protected : The methods or data members declared as protected are **accessible within the same package or subclasses in different packages.**

Exception

Difference between Checked and Unchecked Exceptions

Exception handling helps your program fail gracefully instead of stopping unexpectedly.

1) Checked Exception

The classes that directly inherit the Throwable class except RuntimeException and Error are known as checked exceptions. For example, IOException, SQLException, etc. Checked exceptions are checked at compile-time.

2) Unchecked Exception

The classes that inherit the RuntimeException are known as unchecked exceptions. For example, ArithmeticException, NullPointerException, ArrayIndexOutOfBoundsException, etc. Unchecked exceptions are not checked at compile-time, but they are checked at runtime.

3) Error

Error is irrecoverable. Fz ors are OutOfMemoryError, VirtualMachineError, AssertionError etc.

Java Exception Keywords

Java provides five keywords that are used to handle the exception. The following table describes each.

Keyword	Description
try	The "try" keyword is used to specify a block where we should place an exception code. It means we can't use try block alone. The try block must be followed by either catch or finally.
catch	The "catch" block is used to handle the exception. It must be preceded by try block which means we can't use catch block alone. It can be followed by finally block later.
finally	The "finally" block is used to execute the necessary code of the program. It is

	executed whether an exception is handled or not.
throw	**The "throw" keyword is used to throw an exception.**
throws	**The "throws" keyword is used to declare exceptions. It specifies that there may occur an exception in the method. It doesn't throw an exception. It is always used with method signature.**

throw (used inside method)

✔ Purpose:

Used to **explicitly throw an exception**

👉 You are creating and throwing the exception manually

throw is used to explicitly throw an exception inside a method, whereas throws is used in the method signature to declare that the method might throw exceptions, allowing the caller to handle them.

Wrapper class

The **wrapper class in Java** provides the mechanism *to convert primitive into object and object into primitive*.

Java StringBuffer Class

Java StringBuffer class is used to create mutable (modifiable) String objects. The StringBuffer class in Java is the same as String class except it is mutable i.e. it can be changed.

No.	StringBuffer	StringBuilder
1)	StringBuffer is *synchronized* i.e. thread safe. It means two threads can't call the methods of StringBuffer simultaneously.	StringBuilder is *non-synchronized* i.e. not thread safe. It means two threads can call the methods of StringBuilder simultaneously.
2)	StringBuffer is *less efficient* than StringBuilder.	StringBuilder is *more efficient* than StringBuffer.
3)	StringBuffer was introduced in Java 1.0	StringBuilder was introduced in Java 1.5

1. While creating strings using string literals,

```
String example = "Java";
```

Here, we are directly providing the value of the string (`Java`). Hence, the compiler first checks the string pool to see if the string already exists.

If the string already exists, the new string is not created. Instead, the new reference, `example` points to the already existing string (`Java`).

If the string doesn't exist, a new string (`Java`) is created.

2. While creating strings using the new keyword,

```
String example = new String("Java");
```

Here, the value of the string is not directly provided. Hence, a new `"Java"` string is created even though `"Java"` is already present inside the memory pool.

`Arrays.toString()`

method converts the array `arr1` into a string.

For example, if `arr1` is an array of integers `{1, 2, 3}`, `Arrays.toString(arr1)` will return the string "`[1, 2, 3]`".

//conversion of string to char and char to string

```java
System.out.println("Try programiz.pro");
String arr[]={"kuhu","paro","k"};
//converting array to string
String arr2=Arrays.toString(arr);
//converting string array
char newArr[]=arr2.toCharArray();
for(char c:newArr){
    System.out.println(c);
}
System.out.println(arr2);
//adding array values in list
ArrayList<String> ls=new ArrayList<String>();
for(String x:arr){
    ls.add(x);
}
System.out.println(ls);
```

Name=”kuhu”

char namChar[]=name.toCharArray()

The `ArrayList` class is a resizable array, which can be found in the `java.util` package.

The difference between a built-in array and an `ArrayList` in Java, is that the size of an array cannot be modified (if you want to add or remove elements to/from an array, you have to create a new one). While elements can be added and removed from an `ArrayList` whenever you want.

Getting size:

Arralylist: `System.out.println(list.size());`

`String: List.length()`

`Array:list.length`

`ArrayList=list.size();`

Linked list:made up of nodes data+link

No fixed size

Time complexity

O(n)->searching

O(1)->insert

Collection Framework:

Array is fixed in size,once it is created we can not change in array

```
public static void main(String[] args) {
    System.out.println("Try programiz.pro");
    int a[]={3,6,7};
    int n=a.length;
    int b[]=new int[n+1];
    for(int i=0;i<n;i++){
        b[i]=a[i];

    }
    //now a is pointing to b loaction
    a=b;
    for(int x:a){
        System.out.println(x);
    }
```

When we want variable size at that time we can use Arraylist,LinkedList,set(unique elem)

Collection: group of elements

List: elements are ordered, can have duplicates,ArrayList,Linked,vector,stack

Set: no duplicates(Hash,LinkedHash),no order. Mostly use for searching

Queue: FIFO

LinkedList: to use for insertion cause time complecxity(O(1)). For search(O(n)

ArrayList: best for searching cause time complecxity(O(1)).For insert O(n)

HashMap and Hashtable both are used to store data in key and value form. Both are using hashing technique to store unique keys.

HashMap vs HasTable:

Both look similar

Hashmap : is not thread safe it is not synchronized,allows one null key but multiple null values,fast, Hashtable is **traversed Iterator**.

HashTable: better to use for multithreading application ,it is thread safe,does not allow any null key,values ,slow, Hashtable is **traversed by Enumerator and Iterator**.

HashSet: it stores the unique element

Multithreading

Program → instructions stored on disk

Process → running program with its own execution environment JVM → runs a Java application inside a process T

Thread → unit of execution inside a process Main Thread → initial application thread

Multithreading → multiple threads within one process

Concurrency → multiple tasks make progress during overlapping periods
Parallelism → multiple tasks execute simultaneously

Context Switch → CPU switches execution from one thread to another
Multitasking → multiple processes;

Multithreading → multiple threads

More than one task in a single application . one task called thread.

Video+comment+ad

Thread: smallest sequence instruction executed by cpu independently

1.Thread class(when we extends from single class)

2.Runnable interface(multiple inheritance)

run method we use to overriding working on thread simultaneously

start use to start the thread

Yeild(): causes the current execution thread to pause to allow other threads to execute.

Wait😵() it causes the thread to wait until another thread invokes the notify of this object.it release the monitor lock otherwise deadlock can occur, always need to use inside synchronized block, outside will throw error

Sleep vs wait: sleep will wait for a particular time while wait will be waiting until a notify get invoked

Spurious wake up : thread controll by os and cpu so it might happen if a thread is in wait state for long period it can come to runnable without notify. Which is rare so for this we use while loop.

Notify(): wake up a single thread which was waiting on the object monitor.(1 producer thread 1 consumer thread)

NotifyAll(): it wakes up all thread which are in a waiting state ,better to use this when you use multiple threads for each each task(Ex 2 producer thread 2 consumer thread)

Deadlock : Two or more threads are waiting for each other forever, so none of them can continue.

If there are two synchronized methods and the consumer acquires the lock first, it may get stuck in a while loop because there is no value available. Since the consumer is holding the lock while looping, the producer cannot acquire the lock to produce a value. As a result, the producer waits indefinitely, and the consumer also keeps waiting for the producer, which can lead to a deadlock.

Consumer → gets lock 🔒 → box empty → wait() → releases lock 🔓 ↓

Producer → gets lock 🔒 → produces value → notify() → releases lock

Interrupt()->it send signal to thread to stop whatever state it is in

```
//let t1 finish first then at that time main thread will be on waiting state
//Main thread--> W aiting
//t1 thread --> Runnable->terminated
//Main thread ->Waiting->Runnable
t1.join();
//waiting for 2s only  main only wait for this duration then main will be on runnable state from waiting
//t1.join(2000);


//JVM will not wait for demon thread to get finishes
//better to user for clean up activity,monitoring
```

```
//by default all the threads are user thread
t.setDaemon(true);
```
Purpose: Daemon threads are designed to support user threads and run in the background.

Lifecycle:

If all **user threads** are terminated, JVM automatically terminates all daemon threads, even if they are still running.

JVM **does not wait** for daemon threads to finish their execution.

2. Default Behavior:

All threads are **User Threads** by default.

You need to explicitly set a thread as daemon using:

Atomicity: Either the entire operation completes successfully, or none of it takes effect.

```
//do all or do nothing either thread finishes whole work or nothing
AtomicInteger count = new AtomicInteger(0);
```

- **Visibility problem** → Thread doesn't see another thread's latest value.
- **Reordering problem** → Operations appear to be observed in an unexpected order.
- **Atomicity problem** → An operation gets interrupted/interleaved and produces an incorrect result.
- Its a non locking thing

Synchronized:

- We always lock the object not methods .
- is used to provide access to sharable resources by allowing one thread to access at a time.
- It locks the object if that object have 2 thread even though they are calling different method if one 1 thread already lock 1 method other thread will util get finishes but if you create different object then it can run parallel as are calling different method
- It will make sure 1 thread execute a block of code at a time(think it as a lock if thread 1 started it will lock so other thread can not start it)

- It makes our application slow we don’t use every place

```
static void increment() {
   //class level locking
   //per class will get 1 lock
   synchronized (CounterStatic.class) {
```

Producer consumer problem:

Prodecer produce the value alread but cousmer didnt read it

Consume alread run and check in box there no value prsent because till now producer thread havent got chance

- When you call t1.run(),
  → it just executes like a normal method inside the **main thread**.
  → no multithreading happens.
- When you call t1.start(),
  → JVM creates a **new thread** (Thread-0)
  → then that thread internally calls your run() method.
  → this is how **real multithreading** starts!

💡 Important Notes:

1. You can call run() multiple times (just a normal method).
2. You can call start() **only once per thread object** —
   if you call it again → you'll get

java.lang.IllegalThreadStateException

3. start() is the correct way to begin a thread.

Reentrant lock:

- Here we do manual lock and unlock
- It gives one feature of tryLock thing if get lock do it otherwise perform else block
- IsFair()
- One thread can lock multiple times

```
ReentrantLock lock = new ReentrantLock(true);
```

true means **fair lock**.

Now imagine:

Thread A → holding lock
Thread B → waiting first
Thread C → waiting second

When A releases:

A releases
↓
B gets lock
↓
B releases
↓
C gets lock

Object-Level Lock:
Each object has its own lock.

Class-Level Lock:
There is a single lock for the entire class, shared by all objects.

Shared lock:

- **Here multiple reader thread can enter to a read lock**
- **But if a thread accuire a writer lock then no other can enter**

Stamped lock:

Similar to a shared/read lock, but it provides an additional **optimistic read** feature. Optimistic reading gives us a **stamp**, which we can later use with validate() to check whether another thread acquired a write lock and changed the data while we were reading.

Semaphore:

Semaphore controls how many threads can access a resource at the same time. It uses a number of permits. Multiple threads can enter simultaneously as long as permits are available.

Semaphores can be used to implement rate limiting, controlling the number of tasks processed concurrently.

CAS / lock-free:

"I'll try to update the value. If someone changed it before me, I'll retry."

synchronized:

"I'll acquire the lock. Everyone else waits until I'm done."

ReentrantLock:

"I'll acquire a lock, but I want more control over how I acquire and release it."

Condition:

We can have multiple waiting queue here awaits,signal,signalAll()

volatile vs Atomic

- volatile
- Guarantees visibility between threads.
- A thread reads the latest value of a volatile variable.
- Does not provide atomicity for compound operations.
- count++ is not atomic, even if count is volatile.
- No CAS/retry mechanism.
- Use when you mainly need visibility, not atomic read-modify-write operations.
- Typical use: flags such as shutdown, running, etc.
- Atomic
- Provides visibility + atomic operations.
- Uses CAS for many operations.
- Supports atomic read-modify-write operations.
- Example operations:
- Increment
- Decrement
- Add
- Compare-and-set
- Failed CAS can retry.
- Use when multiple threads need to safely modify shared state.

Lock-Free Retry Loop
- Methods such as AtomicInteger.addAndGet() internally use a CAS retry loop.
- What happens?
- Read the current value.
- Calculate the new value.
- Try CAS(oldValue, newValue).
- If CAS succeeds → operation is complete.
- If CAS fails → another thread changed the value.
- Read the latest value again.
- Recalculate.

- Retry CAS.
- Continue until successful.
- Lock-free doesn't mean "never retry": It means threads don't acquire a traditional lock; they optimistically attempt the update and retry if they lose the race.

AtomicReferenceArray
- Provides an array of atomic object references.
- Each array element can be accessed/updated atomically.
- Supports operations such as:
- get
- set
- getAndSet
- compareAndSet
- Useful when multiple threads work with different elements of a shared array.
- AtomicReference → one atomic reference.
- AtomicReferenceArray → multiple atomic references in an array.

ABA Problem
- What is the issue
- ABA is a potential problem with CAS.
- Initial value is A.
- Thread A reads A and gets paused.
- Thread B changes A → B.
- Thread B later changes B → A.
- Thread A resumes.
- Thread A performs CAS expecting A.
- CAS sees the current value is A → CAS succeeds.
- But Thread A doesn't know that the value actually changed in between.
- Core Problem: CAS checks the current value, not necessarily whether the value changed and then changed back.
- Solving ABA
- Attach a version/stamp to the value.
- Every modification changes the version.
- CAS checks both:
- Value
- Version
- Even if the value returns to its original state, the version will be different.
- Therefore CAS can detect that the value changed in between.

What is ExecutorService?

ExecutorService is an interface in **java.util.concurrent** that provides a **higher-level replacement for managing threads manually** using Thread class.

Instead of creating and starting threads one by one, you give tasks to the ExecutorService, and it handles thread creation, scheduling, and execution for you.

I prefer Runnable because it provides better flexibility and follows good design principles.

1. Runnable

Used when you want to execute a task **without returning a result**.

2. Callable

Used when you want to:

- return a value
- throw checked exceptions

Runnable

public void run()

- does not return anything

Callable

public V call() throws Exception

- returns value
- can throw checked exception

Future in Callable

When you submit a Callable, Java returns a Future.

Future<Integer> future = service.submit(task);

future.get() waits and retrieves the result.

- **Runnable → Runs task**
- **Callable → Calls and returns result**

Data paralisam(ForkJoin): where given task is splited until it reaches to it's least size and executes those task in paralal.

CompletableFuture

This is non blocking unlike future.get () here we use thenAccept which doest not block our main thread.here we can do chaining also which was not possible in future

SupplyAsync->herewe give task

Thenapply->it transform the result

Thenaccept->it takes an cosumer and does not return anything

CompletableFuture is a class introduced in Java 8 used for:

- asynchronous programming
- multithreading
- running tasks in background without blocking the main thread

Execute() vs submit()

- **execute() → does NOT capture the exception** ,it accepts runnable, basically use for fire and forget task
- **submit() → captures the exception inside Future** , it acccpets runnable and callable both, use whree we are expecting a result .
- submit() lets you retrieve it using future.get() as an ExecutionException.

ThreadLocal:

ThreadLocal is used to store data that should belong to the current thread/request, so multiple threads don't interfere with each other's values and we don't have to pass that contextual data through every method.

Whever we create a thread a thread get created in OS whoch is bit expensive beacuse each thread took 512 kb – 1mb that is the reason we use executor but here also concurrency issue is there so here virtual thread can help where 1 to many mapping work many virtual thread connect to singe os thread

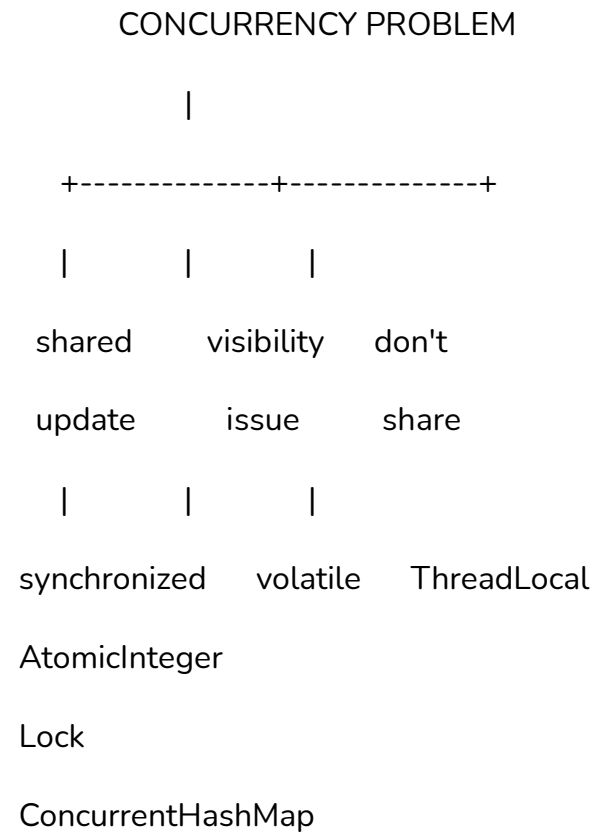
Virtual Thread :

The main thread doesn't automatically wait for that virtual thread to finish.

But you **can absolutely wait** for a virtual thread:

By doing this

```
Thread vt = Thread.startVirtualThread(() -> { System.out.println("Hello"); });
vt.join();
```



Concurrency is the ability to handle multiple tasks in overlapping time periods, even if they are not executing simultaneously.

When should we use asynchronous instead of synchronous?”

Synchronous communication is used when an immediate response is required, such as login or fetching data. Asynchronous communication is used for long-running or background tasks like sending emails, notifications, or processing orders where the client does not need to wait for the result.

Publisher sends messages to a topic, and subscribers listen to that topic and receive the messages asynchronously.

Authentication	Authorization
Verifies **who you are**	Verifies **what you can access**

“When should we use throws instead of try-catch?”

You can answer:

try-catch is used when we want to handle the exception in the same method.
throws is used when we want to pass the exception to the calling method so that it can handle it.

Yes, throw can be used without throws if the exception is an unchecked exception (a subclass of RuntimeException). For checked exceptions, Java requires us to either handle the exception using try-catch or declare it using the throws keyword in the method signature.

Functional interface:

An interface which have only one abstract method but it can have any number of default or static ,methods.

They are intended to use with the lymbda expression

Method Reference = shortcut for simple lambdas that only call a method.

Optional: Uses of Optional like value can be present and not also i mean it can be null also.

New posts

When to use?
✅ Want shared data → Shallow Copy
✅ Want independent objects → Deep Copy

#Java #CoreJava #JavaDeveloper #Programming #OOP

CORE JAVA

Shallow Copy vs Deep Copy



Shallow Copy

- Copies object reference, not actual object
- Both variables point to SAME object
- Change in one → reflects in other

Java

```java
int[] a = {1, 2, 3};
int[] b = a;    // shallow copy
b[0] = 99;

// a[0] is also 99 now!
```



Deep Copy

- Creates completely NEW object
- Both variables are independent
- Change in one → no effect on other

Java

```java
int[] a = {1, 2, 3};
int[] b = a.clone();  // deep copy
b[0] = 99;

// a[0] is still 1
```

When to use:

Want shared data? Shallow copy

Want independent data? Deep copy

```
class threadexample extends Thread{
    //run is the starting of therad
    //we are  overiding the run method
    public void run(){
        int i=1;
        while(i<5) {
            System.out.println(i+"hello");
            i++;
        }
    }
}
//we can extend this classs also from thread
public class Threading {


    public static void main(String[] args) {
        //Threading t=new Threading();
        threadexample t=new threadexample();
        //to start the thread
        //built in method
        //it calls the run method as thread
        t.start();
```

Runnable interface

```
package Multithreading;
class run implements Runnable{
    public void run() {
        int i=1;
        while(i<5) {
            System.out.println(i+"hello");
            i++;
        }
    }
}
public class Runnableinter {

    public static void main(String[] args) {
        // TODO Auto-generated method stub
        run r=new run();
        Thread th=new Thread(r);
        th.start();
```

Lymabda expression: Lambda expressions are mainly used to enable functional programming in Java and are widely used with streams and collections.

We can not modifed the value of that variable which is local to that method

But we can modified the value of class variable

Stream: a sequence of element which support various operation

Parrallel stream **doesnot garantied the order of excution,it uses multiple thread, for large data set it is better to use . We can make it sequencial also by using .sequential()**

Intermediate operations & Terminal operations.

Intermediate Operations

They are lazy they do the execute until a terminal operation is invoked

These operations transform the elements of a stream and return a new stream as a result.

Intermediate operations do not produce a final result directly. they are usually combined using method chaining.

Examples of some common intermediate operations are **filter, map, sorted, distinct, limit, flatMap, skip**, etc.

Terminal Operations

Terminal operations produce a result such as a value or a collection.

Examples of some common terminal operations are **collect, forEach, reduce, count, anyMatch, allMatch, noneMatch, etc.**

🔴 Fail-Fast
A Fail-Fast iterator immediately throws a
ConcurrentModificationException
if the collection is modified while iterating.
👉 It works on the original collection
👉 Stops execution instantly
👉 Helps detect bugs early

Example:
List<String> list = new ArrayList<>();
list.add("A");
list.add("B");

for (String s : list) {
list.add("C"); // ❌ ConcurrentModificationException
}

Why Fail-Fast?
To prevent unpredictable behavior and catch errors quickly.

🟢 Fail-Safe
A Fail-Safe iterator does NOT throw an exception
if the collection is modified during iteration.
👉 It works on a copy (snapshot)
👉 Safe for multi-threaded environments
👉 Allows modifications while iterating

Example:

CopyOnWriteArrayList<String> list = new CopyOnWriteArrayList<>();
list.add("A");
list.add("B");

for (String s : list) {
list.add("C"); // ✅ No exception
}

Strings live in the String Pool.
If they were mutable, changing one reference would affect many others.

Concurrent hashmap:

Thread safe, better for mutithreadd appliaction, synchonized,does not allow any null,key and value.

It does segment base locking (small task division), bydefault it have 16 segment,

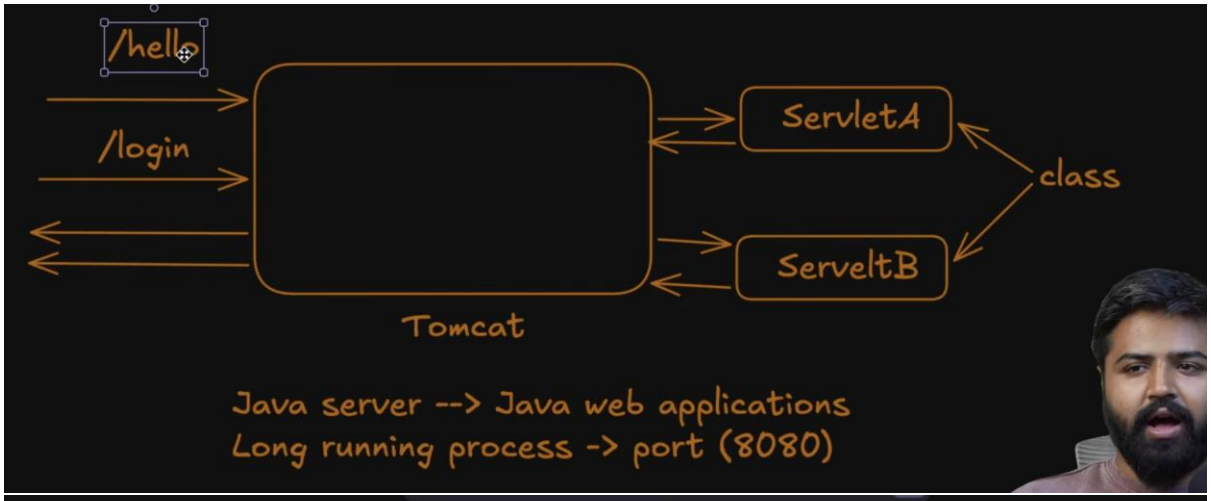
ConcurrentHashMap<String,Integer> map=new ConcurrentHashMap<>();

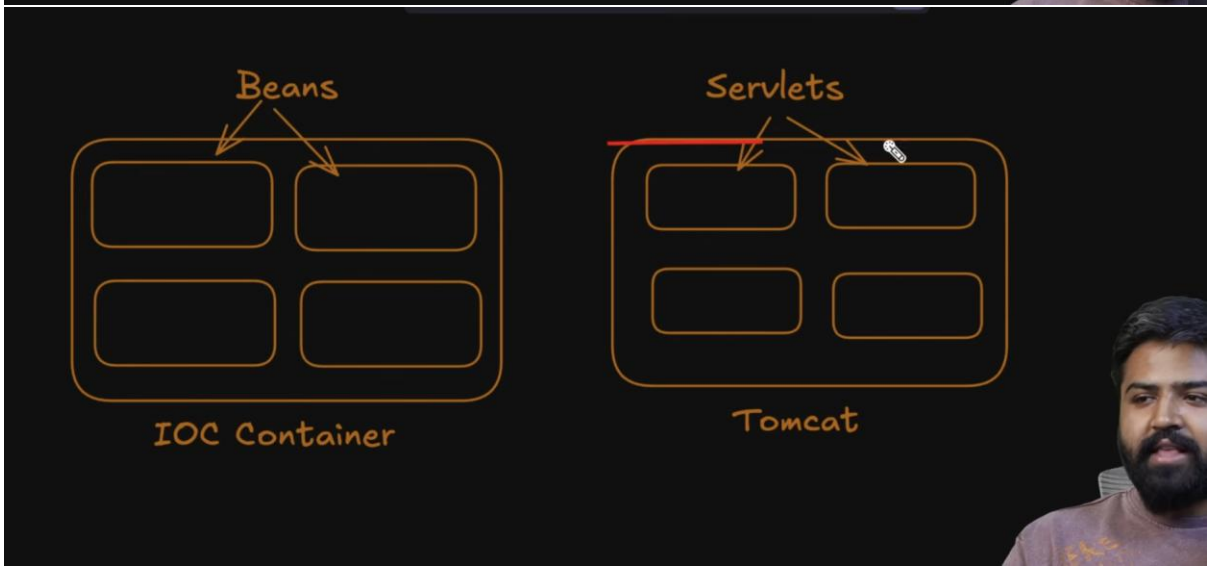
Map.put("kuhu",9);

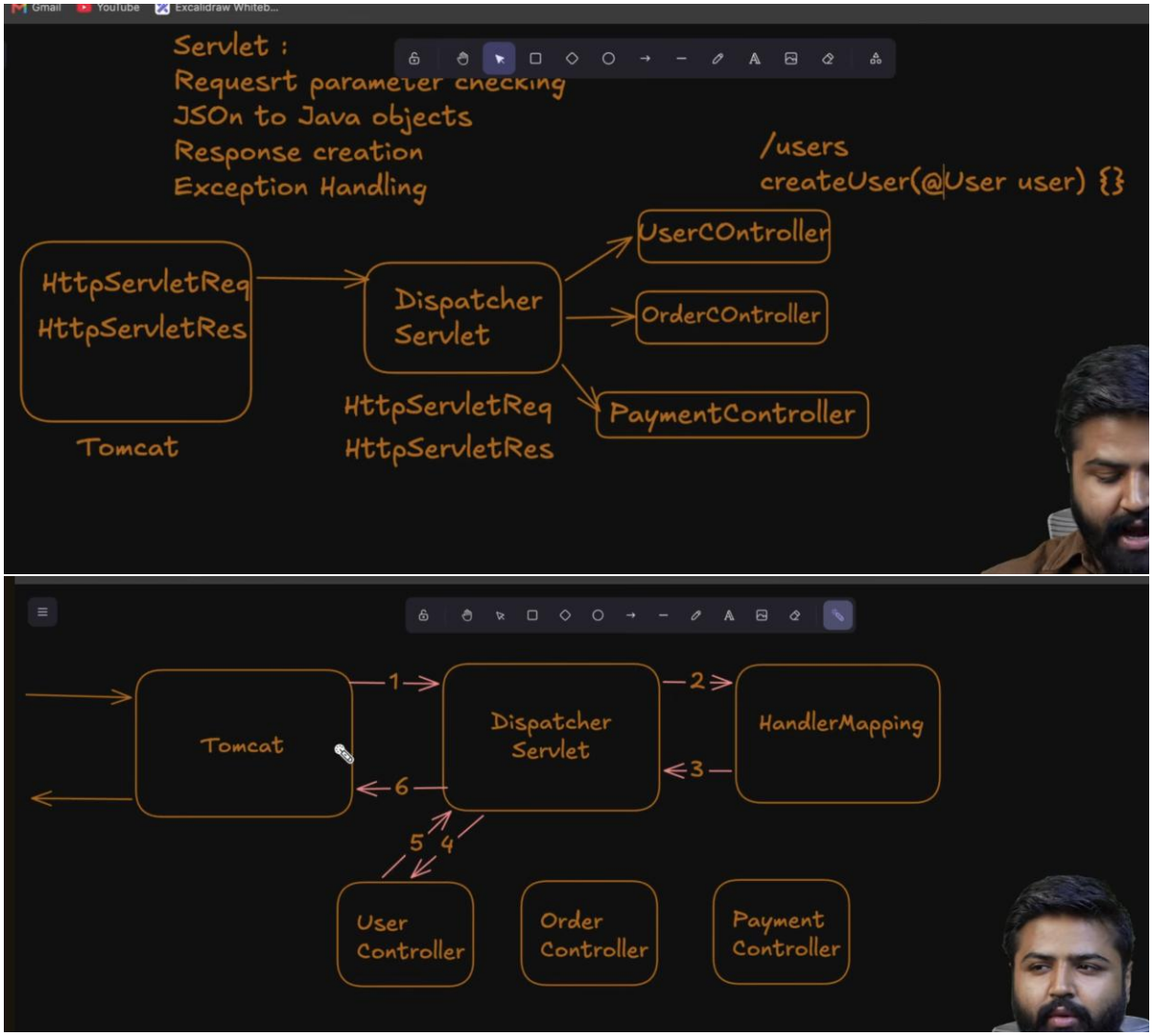
ConcurrentHashMap performs segment locking in a multithreading environment. It allows performing read operations concurrently without any blocking of threads.
It performs retrieval operations without blocking but write operations may conflict each other if they are working on the same segment. Each segment works as an individual HashMap. So, if one thread is writing onto one segment, it locks that segment. Now until that thread is done, no other thread will get access to that segment. However, reader threads can still access this segement. If another thread, wants to write to another segment at the same time, it is free to do so. It will acquire lock on that segment and will start doing its job.
Read operations return most recently updated operations, which means the it may not fetch the currently updating value.
Memory visibility for the read operations is ensured by volatile reads.

HashMap is not thread-safe because multiple threads can access and modify it at the same time without synchronization.
This can lead to problems like:
⚠ Data inconsistency
⚠ Lost updates
⚠ Corrupted internal structure
⚠ Infinite loops during resizing (in older Java versions)

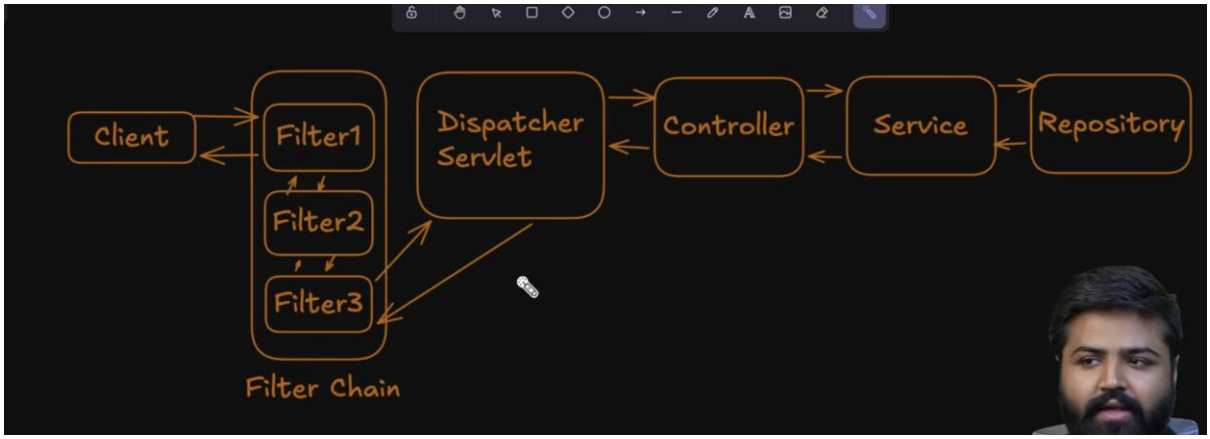
SPRING BOOT STARTED

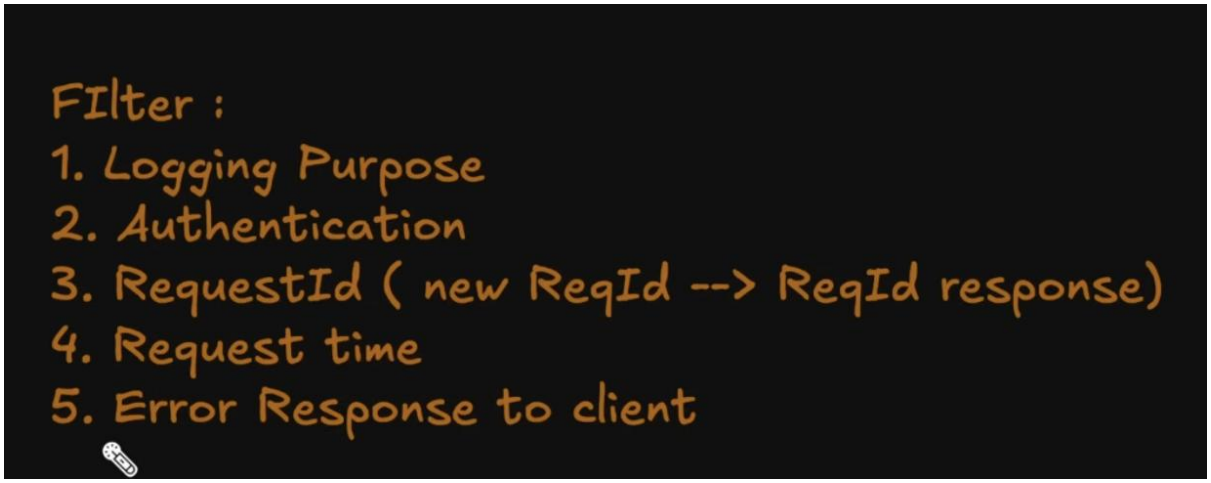






- **DispatcherServlet**
- You can write it like this:
- **DispatcherServlet** is the **front controller** in Spring MVC.
- It receives incoming HTTP requests.
- It uses **HandlerMapping** to find the correct controller and handler method for the request.
- It helps bind **request parameters, path variables, and request body** to Java objects.
- For JSON, it uses **Jackson** to convert JSON → Java object and Java object → JSON.
- It calls the appropriate controller method.
- It processes the controller's response and sends the **HTTP response back to the client**.

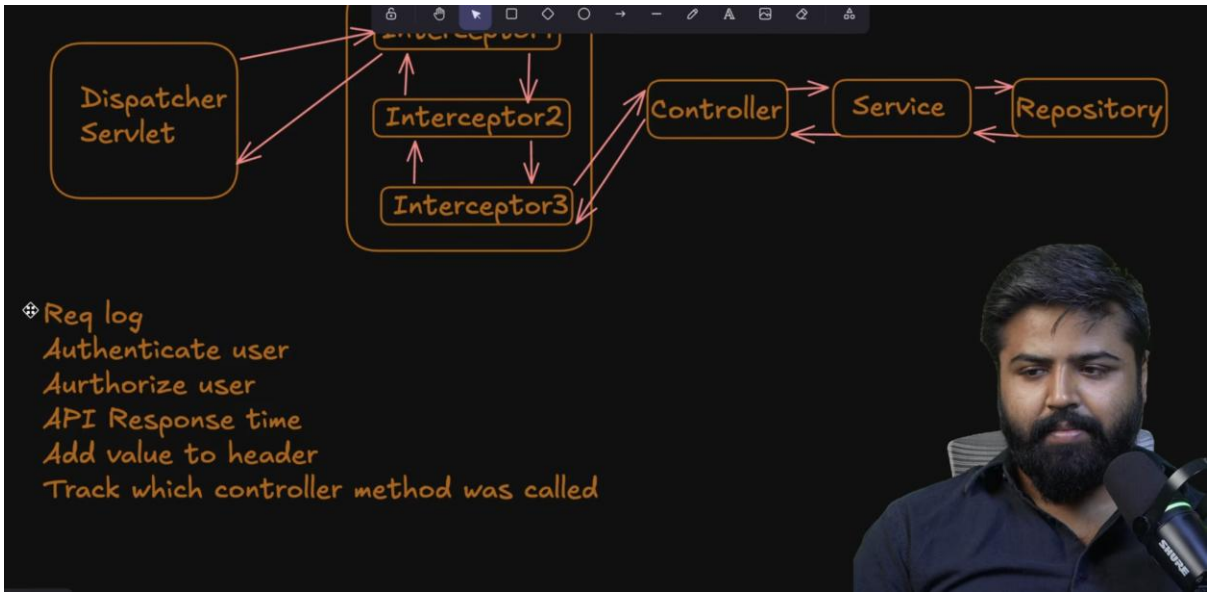





```
@Order(1)
public class AuthenticationFilter implements Filter
```

Order to use which filter will work first

```
//Request log filter->dispatcher->controller
//Response log controller->dispatcher->filter
```



4. Choosing Between Filter and Interceptor

Use a Filter if:

You need to modify all requests, including static files (CSS, JS, images).
You need to work at the raw **HTTP level** (e.g., security, compression).
You **don't need access to Spring beans**.

Use an Interceptor if:

You need to **control requests going to controllers**.
You want to **modify the Spring model or view**.
You need **access to Spring-managed services** (e.g., authentication service).

5. Quick Summary

Filters = Work at the **Servlet** level, modify all requests (including static resources).

Interceptors = Work at the **Spring MVC** level, handle controller-related logic.

Rule of Thumb:

If your task involves modifying raw HTTP requests (headers, encoding, compression) → Use a Filter.

If you need to add logic before or after a controller executes → Use an Interceptor.

Demo.java->(by compiler)demo.class->jvm->output

Jar->package where multile class file,resourrses and pack them in a jar and we can share it to people

Tomcat ->a servlet container which takes the http request from client and give back to the client

Effective pom ->my own pom+ super pom (which is the parent pom)

Maven lifecycle-> validate(validate the folders)->compile(make .java class to .class file)->test(unit test)->package(makea jar file and store it in target folder)->verify(code quality checks using plugins(ex: sonar cube)->install(install in local repo)->deploy(from local to remot repository upload)

Maven : Build automation tool(validate,compile,test etc all this thing mvn will do),Manages the dependencies

From cmd to run spring application:

Mvn package

Java –jar jaName(fat jar)

A **Fat JAR** is a JAR file that contains:

- Your Java code

- PLUS all required **dependencies**
- PLUS resources

Mvn clean ->it clean the target folder not our local repository(.m2 foolder our local cache)

Group id -> domain name(com.google)

Artificate ->project name(google image)

Bean: it's a java object which is managed by spring

IOC Container:

it manages all the beans along with dependency of the beans, it will handle the bean life cycle.it is actullay an terminalogy actual implementaion is Application context

IOC(Inversion of control):

Classes which are annotated with component/service etc ioc container will keep those class so when we ask for obj spring can provide it , but we dont use annotation spring will not handle them

Application context: it helps us to achieve IOC through this

Dependency Injection: @Autowired: using this we are asking obj from spring

It's a design pattern

Previously in java we was using new keyword to create obj of different class but that will make our application tighty coupled but with the help of spring we can do this like the below image

Here we will not create obj manullay our spring will do that

This process is called inversal control(IOC)

```
OnlineOrder onlineOrder; 1 usage
@Autowired
public User(Order order, OnlineOrder onlineOrder){
    this.order = order;
    this.onlineOrder = onlineOrder;
    System.out.println("Initializing User");
}
```

```
@Component no usages
public class User {
    Order order; 1 usage

    @Autowired
    public User(Order order){
        this.order = order;
        System.out.println("Initializing User");
    }

}
```

3 types of DI: field,setter,Constructor

Constructor: multiple fileds we can use here, avoids null pointer exception, create immutable object final Order order;

Circular dependancy: when both class are dependant . @postconstruct, @Lazy we can use to avoid circular dependancy

Circular dependency: when 2 beans are dependent on each other that issue, so resolve this @Lazy annotaion is used it means when needed create the bean.

//@Lazy on the constructor parameter tells Spring to lazily resolve that particular dependency
//it create a proxy of that class
@Autowired
*/*this will create proxy of LazyPaymentService not the obj of LazyPaymentService but the moment we will call method from LazyPaymentService spring*

```
will create the object  and remove that proxy this resolves our circular dependency*/
public LazyOrderService(@Lazy LazyPaymentService service){
   this.service = service;
}
```

Dependency Injection means Spring provides the required dependency to a class instead of the class creating that dependency itself.

Spring IoC container manages the creation and lifecycle of beans, and Dependency Injection allows Spring to provide those beans to other beans that depend on them.

Class metadata->class name,field name ,filed type,constructors,method;

```
//this s store metadata of Student class not s is pointing to a object here
//reflection api
Class<Student> s= Student.class;
//@ComponentScan ->it check which classes have annotaion and keeps those in ioc container
//@Configuration ->it telling spring this is  a configuration class

//AppConfig.class->it provides the metadata
// ApplicationContext -> made the  ioc container up reads the rules from AppConfig.class

ApplicationContext context = new AnnotationConfigApplicationContext(AppConfig.class);
OrderService order = context.getBean(OrderService.class)
```

Why construction injection preferred?

- Dependency get wired at the object creation time
- Final variable can be used
- Easy to do Unit test

ApplicationContext is a type of **IoC Container**, not the other way around.

Relationship

IoC Container
|
+-- BeanFactory
|
+-- ApplicationContext

- **IoC Container** is a general concept in Spring that manages the lifecycle and dependencies of objects (beans).
- **BeanFactory** and **ApplicationContext** are implementations of the IoC Container.
- **ApplicationContext** is the most commonly used IoC container because it provides additional enterprise features.

Application context: it is an implementation of spring ioc container,it will store the beans of our application

Bean factory:

/actuator/beans: here we can check what are the beans are created

Bean creation ways:

2 ways to create bean

1,@component,service

1.@configuration+@Bean

Method wise(@bean) where cant use component as like using the libraries their we cant modify the code but we want to use it then in that case we can use of @Bean+@Configuartion annotation,
Class

Qualifier always get preference

// Bean creation ways:
//

// 1. Class-wise → @Component
// Used when we can modify the class and add @Component.
//
// 2. Method-wise → @Bean + @Configuration
// Useful when we cannot modify the class, such as classes
// from third-party libraries, or when we want to control
// how the object is created.

Bean *initialization ways :*

1.eager(by defautt one singleton) 2.Lazy(prototype)

//lazy initialization(means when need that time only create the object
// and singletone is eger initialization by default (means do it as soon as container is coming up
@Scope("prototype")

Prototype(for statefull class ex User which can exist multiple like multiple user details):

- *** Whenever we call getBean(), Spring creates a new object.**
- *** Also, when a prototype bean is injected into a dependency,**
- *** a new instance is created for that injection.**
- *** Spring does NOT create prototype objects automatically**
- *** when the application starts if nobody requests them.**
- **These are by default lazy @Lazy/@protoype scope work same**

Singleton(for stateless class as order service):

- **By default, Spring creates one object for each bean and keeps it in the Spring container.**
- **For singleton beans, the object is normally created when the application starts, even if we haven't explicitly requested it yet.**
- **But if we create multiple bean in Appconfig file at that different bean will get created.**
- **Singletone classe are by default eger one to may it lazy initilization @ Lazy need use**

To make every class Lazy need to make it in properties lazy-initilizaton= true which is by default false and you want class not be lazy(@Lazy (value= “false”)

```
// Best candidate for prototype scope

public class User {  no usages
    private String name;  no usages
    private int age;  no usages
}

// states of user --> stateful
```

Singletone bean: one time it will get created and every where it is going to use

@Scope(“prototype”): lazy initialization example every time a new object will created

- Lazily initialized - New Object is created each time - created when needed

Request:

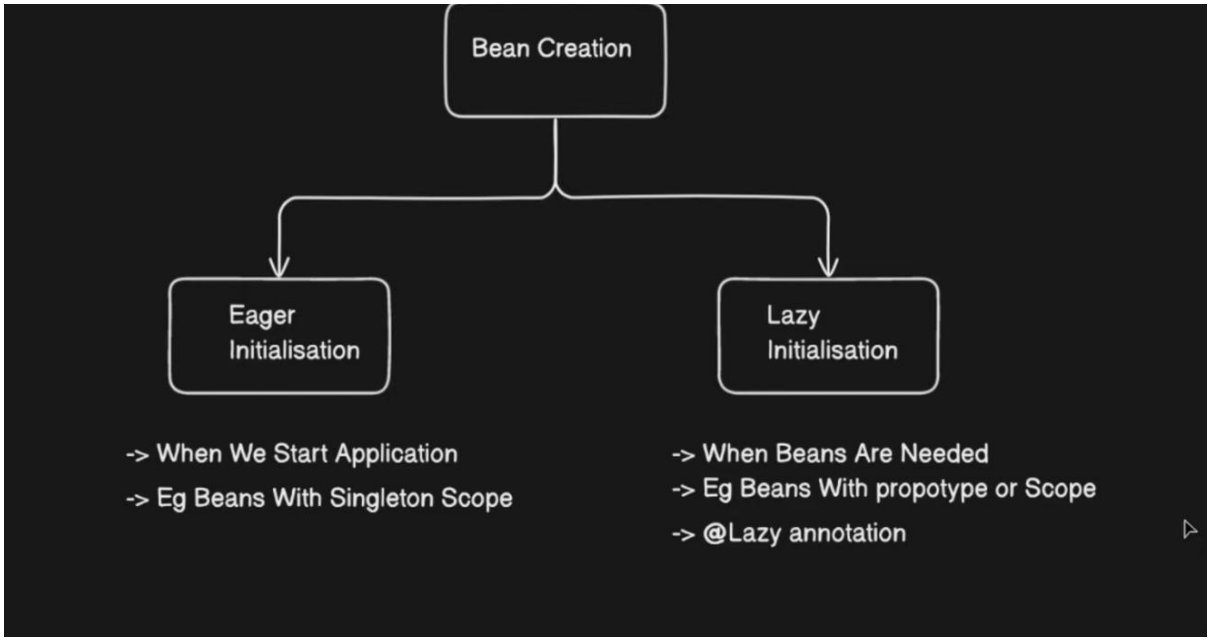
- One bean per HTTP request
- Lazily Initialized
- proxyMode - injecting request scoped bean in singleton scoped bean

Session

- New Object for each http session
- Lazily Initialized
- When user access any api, session is created
- Remains active till session is expired

How spring finds beans:

@ SpringBootApplication if we go inside this @componentscan is the this helps to find beans



Bean life cycle:

Application started->ioc container (web application context)will started then it will will construct the bean it will does the scanning and beans will get created then inject the depedencies add the dependcies into the constructed beans means whch is just got created then do postconsurt means affter bean creation if we want do somethoing, use bean the pre destroy

Spring Bean Lifecycle(Singletone bean)

1. **Application Starts**
    a. When the Spring Boot application starts, the **IoC Container / Application Context** gets initialized.
2. **Bean Scanning**
    a. Spring scans classes using annotations like:
        i. @Component
        ii. @Service
        iii. @Repository
        iv. @Controller

   Read bena defination (classe metadata)
   Bean name
3. **Bean Creation**
    a. Spring creates objects (beans) for the detected classes.
4. **Dependency Injection**
    a. Spring injects required dependencies into the bean using:
        i. Constructor Injection
        ii. Setter Injection
        iii. Field Injection (@Autowired)

Aware interface got called : spring call some method as as example set spring bean name

5. **Initilization callback(Init method/@PostConstruct)**
    a. After dependency injection is completed, methods annotated with @PostConstruct are executed.
    b. Used for initialization logic. Like cache you want clear and some work start from scarch as eaxpme use a map but first clear that map then start

Example:

```
@PostConstruct
public void init() {
   System.out.println("Bean initialized");
}
```

6. **Bean Ready to Use**
    a. Now the bean is fully initialized and ready for use in the application.
7. **PreDestroy** /destroyMethod
    a. Before application shutdown, methods annotated with @PreDestroy are executed.
    b. Used for cleanup activities like:
        i. closing connections
        ii. releasing resources
        iii. stopping threads

Example:

```
@PreDestroy
public void destroy() {
   System.out.println("Bean destroyed");
}
```

8. **Application Stops**
    a. Spring container shuts down and destroys beans.

@PostConstruct: This method is called after the Spring bean (in this case, ExampleBean) has been created and all dependencies have been injected. It's an ideal place to put initialization logic.

@PreDestroy: This method is called before the Spring bean is destroyed. It's where you can place cleanup logic, such as closing resources.

Prototype bean

But if the scope is prototype then in that case after initiliaztion callabck spring will hand the object to user now spring willl not mange that pre destry will not autimatically garbeg collector will do that or use try with resource the reason is every time protype createa new object so if it store our ioc it will get filled which can lead to memory leak

Simple Flow

Application Start
↓
IoC Container Created
↓
Bean Scanning
↓
Bean Creation
↓
Dependency Injection
↓
@PostConstruct
↓
Bean Ready to Use
↓
@PreDestroy
↓
Application Shutdown

In interviews, you can summarize it like this:

“When the application starts, the Spring IoC container initializes, scans and creates beans, injects dependencies, executes @PostConstruct methods, and when the application shuts down, it executes @PreDestroy methods before destroying beans.”’

@ComponentScan: it scans the packages and instantiate the beans. It comes under @springbootapplication it shoub be use with @Configuration

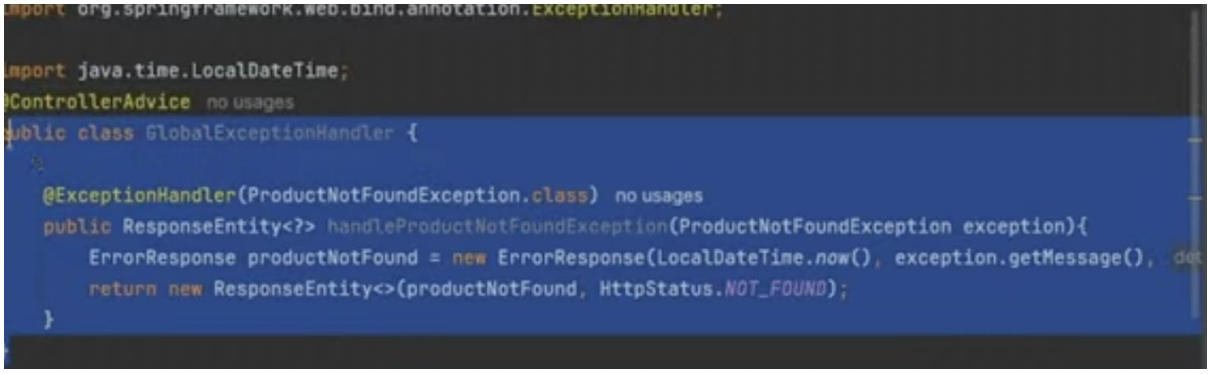
@configuration is always needed other wise bean will not get created

```
import org.springframework.context.annotation.Configuration;

@Configuration  no usages
@ComponentScan(basePackages = "com.codesnippet.ecomassistant")
public class EcomConfig {
}
```

Customize the nature of beans means : after creation of bean we are peroming certain thing as a exmple initialize something, before destorying the bean doing something.

Global exceptional handler:



@controlleradvice+@exceptionalHandler use to handle the exception globally for the all controller,

It is best to use for spring mvc application,

@RestControlleradvice: for springboot application

✅ **Key Differences:**

Feature	`@ControllerAdvice`	`@RestControllerAdvice`
Purpose	Used for global exception handling and applying cross-cutting concerns to controllers.	Specifically used for exception handling in REST APIs.
Applies To	`@Controller` (MVC controllers that return views)	`@RestController` (REST APIs that return JSON responses)
Implicit Behavior	Returns **views (HTML pages)** by default.	Returns **JSON responses** by default (automatically applies `@ResponseBody`).
Common Usage	Used in web applications with **Thymeleaf/JSP**.	Used in **REST APIs** that return JSON responses.

Client sends request to controller. Controller validates input and calls service layer. Service layer contains business logic and interacts with repository layer. Repository uses JPA/Hibernate to fetch data from database. The response is converted into DTO and returned as JSON to the client. We also implemented exception handling, logging, and unit testing for the API.

Spring Core:

It will take care of creation of obj

@SpringBootApplication

=

@**SpringBootApplication**:

@SpringBootConfiguration+ @EnableAutoConfiguration+

@ComponentScan-> go find the bean which spring need to create and mange the dependencies

@SpringBootConfiguration -> This class is the main configuration class of my Spring Boot application. Spring needs some starting point. Spring Boot can start from this class

@EnableAutoConfiguration-> Look at the dependencies I have added and automatically configure the application for me.

Example

Suppose you add:

spring-boot-starter-web

to your project.

This brings many web-related dependencies.

Spring Boot sees thos J e dependencies and says:

"Oh, this application is using Spring Web."

So it automatically configures things needed for a web application.

For example, it can configure:

- Spring MVC ,embedded Tomcat
- SON support
- HTTP message converters
- DispatcherServlet
- etc.

You don't have to configure all of these manually.

"What is @SpringBootApplication?",

"@SpringBootApplication is a convenience annotation that combines @SpringBootConfiguration, @EnableAutoConfiguration, and @ComponentScan. @SpringBootConfiguration marks the main configuration class, @EnableAutoConfiguration lets Spring Boot configure components automatically based on the application's dependencies, and @ComponentScan finds Spring components such as @Service, @Repository, and @Controller."

One subtle but important point: **@ComponentScan finds your components; @EnableAutoConfiguration mainly imports Spring Boot's predefined auto-configuration classes.** They solve different problems.

we just use it in main class . It removes all the need of manual confi which we was doing in spring ex: tomcat

@Component is used for automatic bean creation through component scanning, while

@Bean is used to manually define and configure beans inside a configuration class, especially for third-party or customized objects.

SpringApplication.run()

Here's what happens behind the scenes:

1. Spring creates the ApplicationContext (IoC container)
2. It scans the project for components and configurations
3. Auto-configuration decides which beans should be created
4. An embedded server (usually Tomcat) starts automatically
5. DispatcherServlet is registered as the front controller
6. All beans are initialized and dependency injection is completed

In simple terms:

Spring Boot = IoC Container + Auto Configuration + Embedded Server

This is what allows developers to build production-ready applications with minimal setup.

```java
interface ways{
    public void display();
}
class abc {
    //this one we can modify
    int a=20;
    public void method1() {
        //this value we can not modified through lyambda
        //only we can print this
        int count=9;
        ways w=()->{System.out.println("i am lyambda");
        System.out.println(a);
        System.out.println(++a);
        System.out.println(count);
        };
        w.display();
    }

}
```

Spring framework

Ways to create properties in spring framework

```xml
<!--first student-->
 <bean class="springBootProject.student" name="student1">
 <property name="studentId" value="3455"/>

 <property name="studentName" value="kuhu"/>

 <property name="studentAddress" value="Delhi"/>


 </bean>
 <bean class="springBootProject.student" name="student2">
 <property name="studentId" value="3955"/>

 <property name="studentName" value="nasu"/>

 <property name="studentAddress" value="Blr"/>
 </bean>
  </beans>
```

Autowiring

Automating linking

```xml
<bean class="SpringAutowiring.Address" name="address">
<property name="city" value="delhi"></property>
<property name="street" value="delhi001"></property>
</bean>
<bean class="SpringAutowiring.Emp" name="emp1" autowire="byName">
</bean>
```

Primary/Qualifier

When 2 class implements same interface in that case if we are doing constructor injection it will show error like 2 bean found so need to @Primary to avoid this issue

Both **@Primary** and **@Qualifier** are used when Spring finds **multiple beans of the same type** and doesn't know which one to inject.

```
@Primary
@Qualifier
// @Primary → tells Spring which bean to use by default
// when multiple beans of the same type are available.

// @Qualifier → explicitly tells Spring which specific bean
// to use at the injection point.
public class UpiPayment implements PaymentService{
 And to use qualiofier you have use it like this in camal case
public OrderService(@Qualifier("cardPayment")PaymentService
paymentService){
//@Qualifier("cp")
//changing the  bean name from CardPayment->cp
```

Another thing we camn use @Qualifier

```
@Autowired

public User(@Qualifier("onlineOrder") Order order){
    this.order = order;

    System.out.println("Initializing User");
}
```

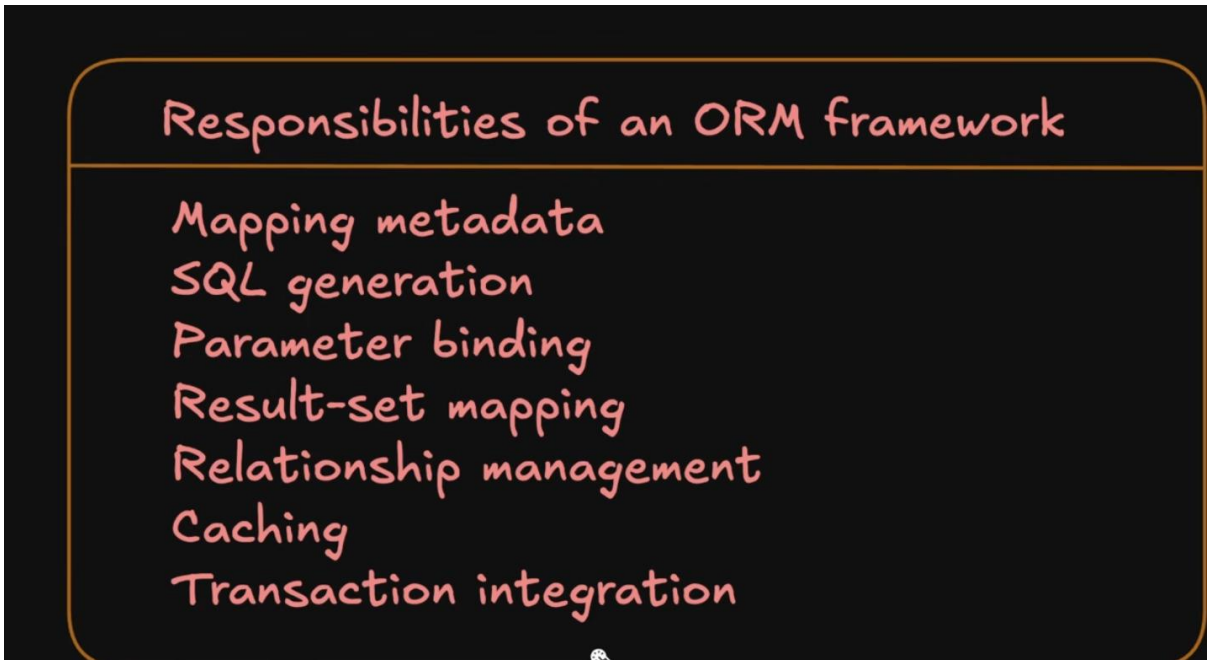
If there is qualifier annotation it will get the priority over primary.

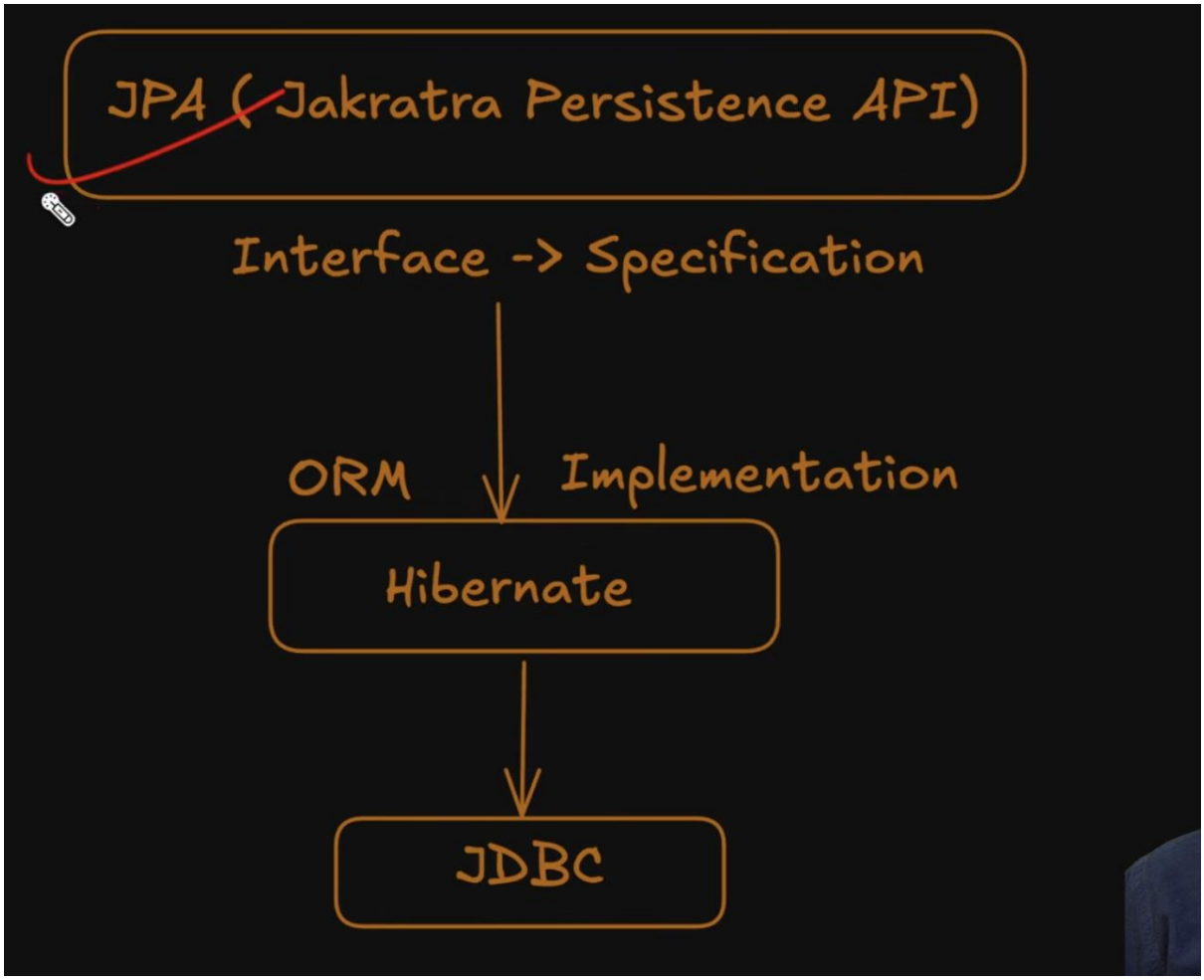
@ConfigurationProperties("gateway-property")//read all the value which my class need less code
@Value("${paymentGateway.type}")//read single value
Using these 2 way we can read value from our properties

```
// it will run automatically when application will start
 //print in console
@Component
public class DemoRunner implements ApplicationRunner
```

Custom Annotation: public @interface TrackExecutionTime
And in method have to use like this @TrackExecutionTime

Spring JDBC





1. JDBC vs Spring JDBC vs Hibernate

1. JDBC (Java Database Connectivity)

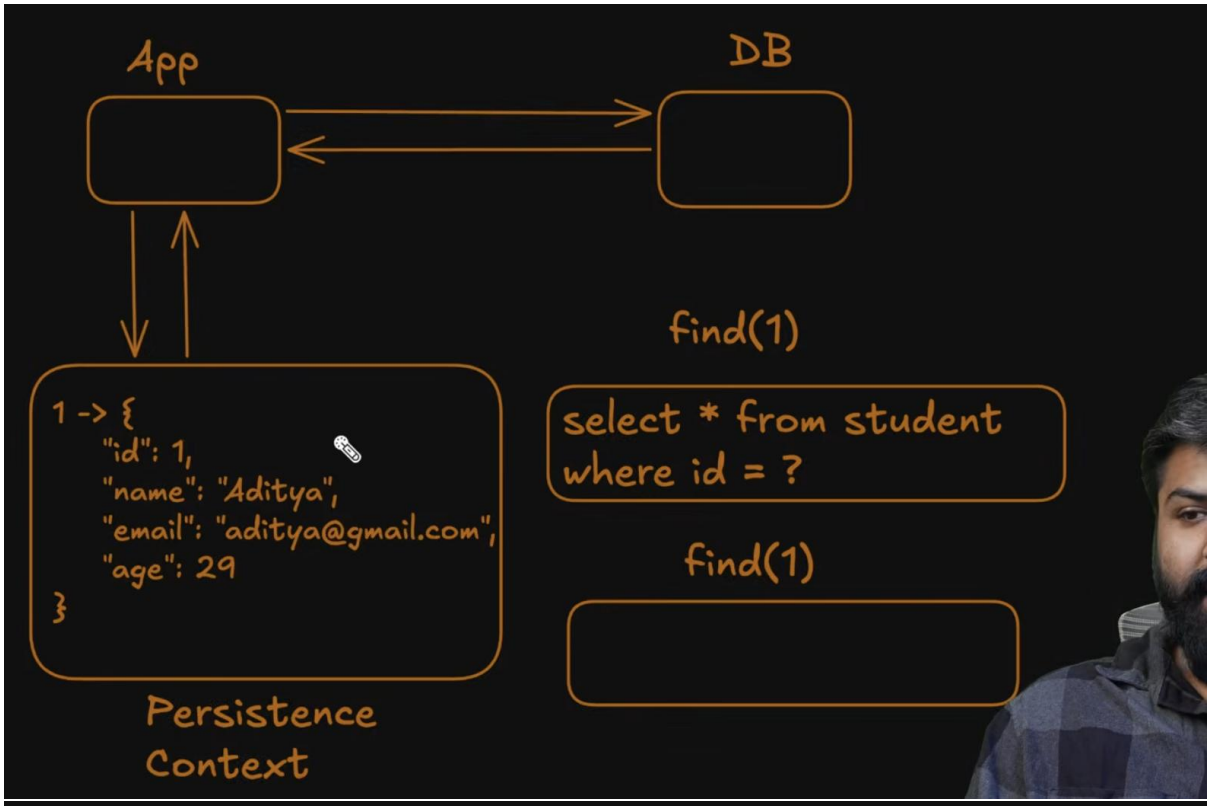
- We manually write SQL queries.
- We need to manage database resources, such as opening and closing connections, statements, and result sets.
- We handle SQLException, which provides database-related error information.
- There is a lot of boilerplate code.

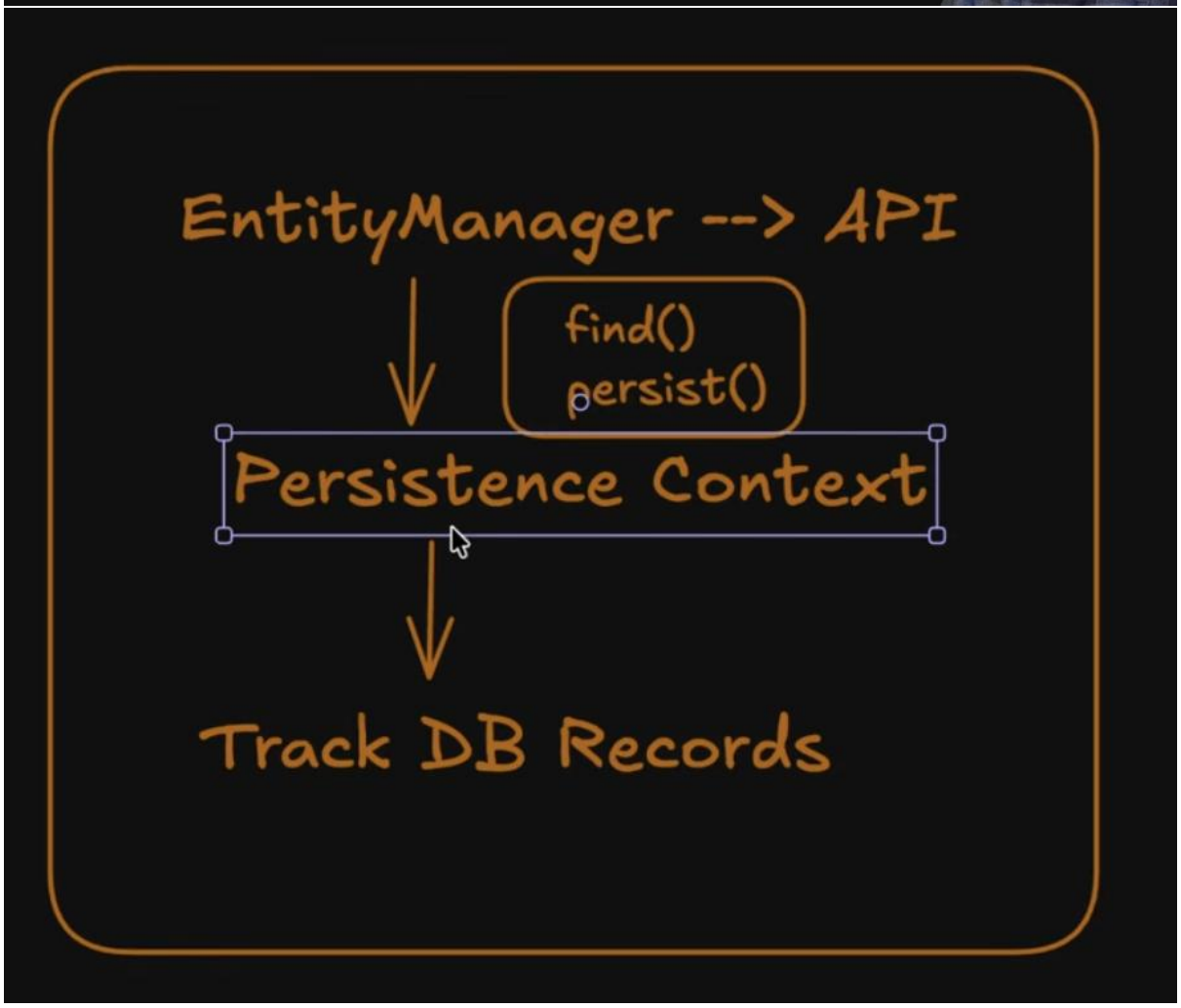
2. Spring JDBC (JdbcTemplate)

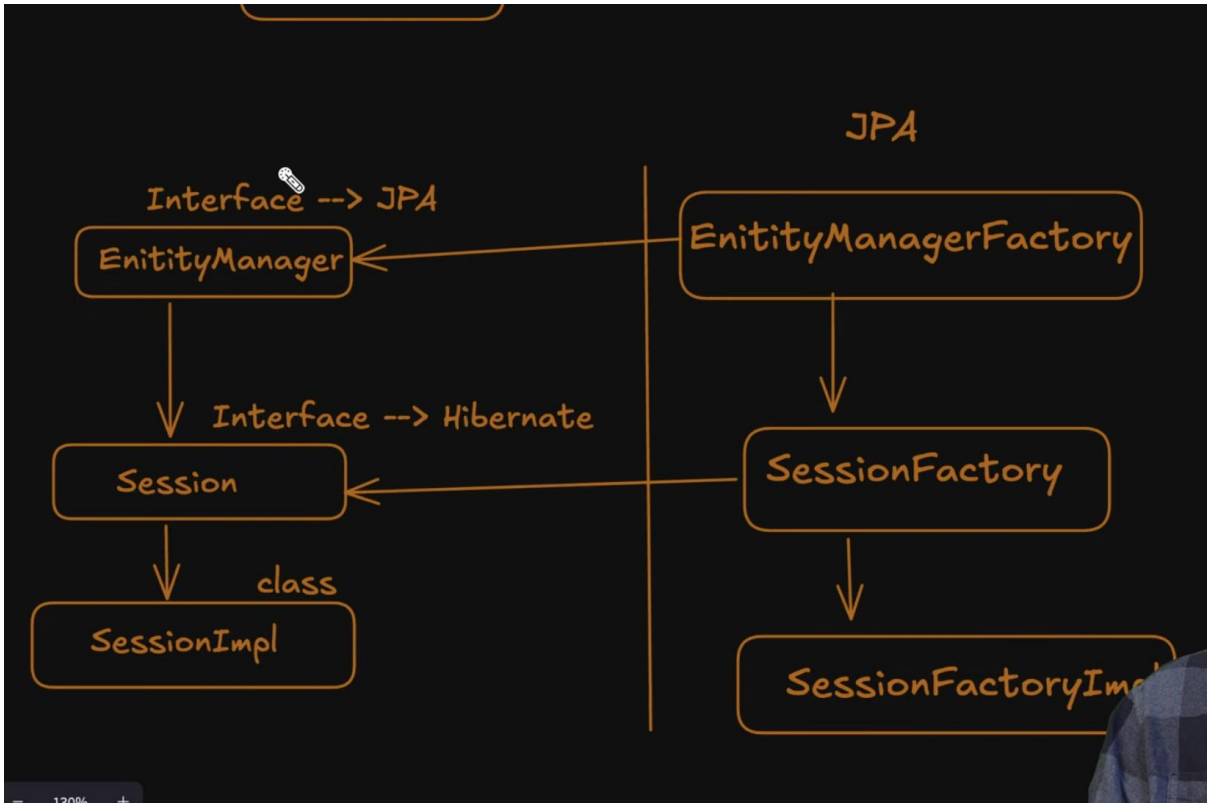
- Spring provides JdbcTemplate to reduce boilerplate code.
- It manages opening and closing connections and other JDBC resources.
- It translates SQLException into Spring's unchecked DataAccessException hierarchy, which provides more specific exception types.
- We still write SQL queries manually.
- We generally map query results to Java objects ourselves, using a RowMapper or similar mechanism.

3. Hibernate (ORM framework)

- Hibernate is an ORM (Object-Relational Mapping) framework.
- It uses JDBC internally to communicate with the database.
- We work with Java objects and entities rather than writing SQL for every CRUD operation.
- Hibernate can generate SQL automatically for operations such as saving, updating, and deleting entities.
- It maps Java objects to database tables and manages entity relationships.
- We can still write queries manually using JPQL/HQL or native SQL when needed.


App
DB
find(1)
1 -> {
"id": 1,
"name": "Aditya",
"email": "aditya@gmail.com",
"age": 29
}
select * from student
where id = ?
find(1)
Persistence
Context


EntityManager --> API
find()
persist()
Persistence Context
Track DB Records



@PersistenceContext and Persistence Context

- @PersistenceContext is used to inject an EntityManager into a class.
- The EntityManager manages entities within a **persistence context**, which acts like a container that keeps track of managed entity objects.
- When a method annotated with @Transactional starts, Spring typically provides an EntityManager associated with the current transaction. It does not necessarily create a completely new one every time.
- When we call find():
    - JPA first checks whether the entity is already managed in the persistence context.
    - If the entity is already present, it can return the existing entity without querying the database.
    - If it is not present, JPA typically fetches it from the database and stores it in the persistence context.
- The persistence context is generally associated with the transaction. When the transaction ends, its entities are no longer managed by that transaction's persistence context.

1. Transient (New)

The object is created using new, but it is not associated with a persistence context.

```
Student student = new Student();
student.setName("Nasima");
```

No database record is created just by creating this object.

Call persist(student)

2. Managed (Persistent)

The entity is associated with a persistence context. Hibernate tracks its changes.

```
entityManager.persist(student);
student.setName("Updated Name");
```

Changes to a managed entity can be synchronized with the database at flush or transaction commit.

Call entityManager.detach(student)

3. Detached

The entity was previously managed but is no longer associated with that persistence context.

```
entityManager.detach(student);
student.setName("Another Name");
```

Changes made after detachment are not automatically tracked or saved.

4. Removed

The entity is still managed, but it has been marked for deletion.

```
entityManager.remove(student);
```

The database deletion normally happens when changes are flushed or the transaction commits.

4. What happens when you call findById()?

Suppose your service contains:

```
@Transactional(readOnly = true)
public Student getStudent(Long id) {
   return studentRepository.findById(id)
         .orElseThrow();
}
```

The process is:

1. Spring starts or joins a transaction as appropriate.
2. Spring Data JPA invokes the repository implementation.
3. Hibernate uses the transaction-associated EntityManager and persistence context.
4. Hibernate checks the persistence context for the entity.
5. If the entity isn't there, Hibernate generally queries the database.
6. The entity becomes managed by that persistence context.

1. How the N+1 problem occurs

Suppose you have two entities: Student and Department.

- Student has a department attribute.
- The department attribute uses FetchType.LAZY.
- You fetch all students first, and then access each student's department.

The N+1 problem occurs when we execute one query to fetch a list of entities and then additional queries to fetch their associated entities individually. For example, if we fetch 100 students and access each student's lazily loaded department, Hibernate may execute one query to fetch the students and additional queries to fetch the departments. This can lead to N+1 queries and degrade performance. We can address it using JOIN FETCH, EntityGraph, or batch fetching.

3. How can we solve it?

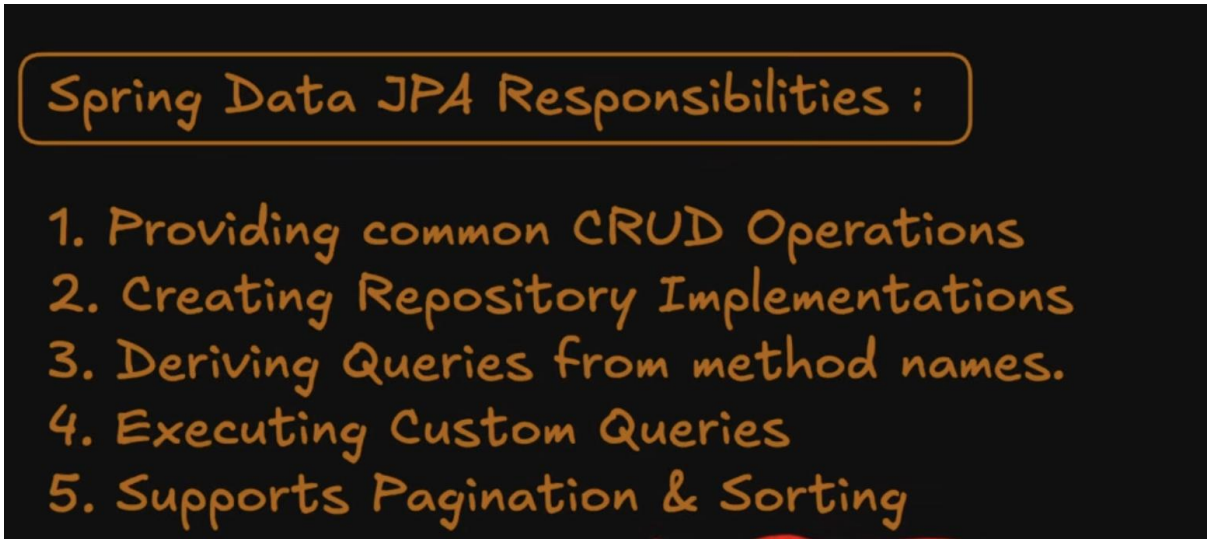
One common solution is JOIN FETCH.

```
@Query("SELECT s FROM Student s JOIN FETCH s.department")
List<Student> findAllWithDepartment();
```

Hibernate can fetch the students and their departments in a single SQL query using a join, rather than executing a separate query for each department.

Other solutions include:

- @EntityGraph — specify which relationships should be fetched.
- Batch fetching — load multiple related entities in fewer queries.
- DTO projections — retrieve only the fields you actually need.



How @Transactional Works Internally in Spring Boot

1. What is @Transactional?

@Transactional is a Spring annotation used to manage database transactions. It allows a group of database operations to succeed together or roll back together if a qualifying failure occurs.

2. Internal Steps of a Transaction

Suppose we have this service method:

```
@Transactional
public Student createStudent(Student student) {
   return studentRepository.save(student);
}
```

Step 1: Spring Transaction Proxy

When another Spring bean calls the transactional method, the call usually goes through a Spring proxy.

The proxy intercepts the call and asks the transaction manager to start a transaction or join an existing one, depending on the transaction configuration.

Step 2: Transaction Begins

Spring's transaction manager manages the transaction.

With Spring Data JPA, Hibernate and the underlying database connection participate in the transaction.

Step 3: The Method Executes

The service method starts executing.

When we call:

studentRepository.save(student);

Spring Data JPA delegates the operation to the JPA EntityManager.

- For a new entity, JPA typically uses persist().
- For an existing entity, Spring Data JPA may use merge().

Step 4: Persistence Context Manages the Entity

Hibernate manages the entity in the persistence context.

The persistence context tracks managed entities and their changes. Hibernate may delay executing SQL until a flush occurs.

Important: With some ID-generation strategies, such as GenerationType.IDENTITY, Hibernate may execute the INSERT during save() itself to obtain the generated ID.

Step 5: The Transactional Method Finishes

When the method returns, control goes back to the Spring transaction proxy.

If this method owns the transaction, Spring proceeds toward completing it.

If the method joined an outer transaction, the outer transaction controls the final commit.

Step 6: Flush

Before a successful commit, Hibernate flushes pending changes to the database.

During flushing, Hibernate executes the required SQL statements, such as INSERT, UPDATE and DELETE.

Flush does not mean commit. The transaction is still capable of being rolled back.

Step 7: Commit

If flushing and transaction completion succeed, Spring's transaction manager commits the transaction.

The database makes the transaction's changes permanent.

3. What Happens When an Exception Occurs?

Consider this example:

```
@Transactional
public void createStudent(Student student) {
   studentRepository.save(student);

   entityManager.flush();

   throw new RuntimeException("Something went wrong");
}
```

The flow is:

1. Spring starts the transaction.
2. The repository saves the student.
3. Hibernate flushes the pending SQL to the database.
4. A RuntimeException occurs.
5. The exception escapes the method.
6. Spring rolls back the transaction by default.

Even if the INSERT was already executed during flushing, the database can undo it because the transaction has not committed.

This assumes the exception escapes the transactional boundary and the transaction has not already been committed.

4. Difference Between Save, Flush, Commit and Rollback

Operation	Meaning
save()	Delegates to JPA to persist or merge an entity.
Flush	Synchronizes pending changes by executing SQL against the database.
Commit	Finalizes the transaction and makes its changes permanent.
Rollback	Undoes the transaction's uncommitted changes.

5. Easy Flow to Remember

Successful transaction:

Spring proxy → Begin transaction → Execute method → save() → Flush → Commit

Failed transaction:

Spring proxy → Begin transaction → Execute method → save() → Flush → Exception → Rollback

The INSERT may execute before the explicit flush, depending on the situation.

6. Interview Answer

When a method annotated with @Transactional is called, the call usually passes through a Spring proxy. The proxy starts a transaction or joins an existing transaction and then executes the method.

When we call repository.save(), Spring Data JPA delegates to the EntityManager, which manages the entity through the persistence context. Hibernate may execute the SQL immediately or delay it until flushing.

When the transactional method finishes, the proxy handles transaction completion. Hibernate flushes pending changes before a successful commit. If a qualifying exception occurs, Spring can roll back the transaction, even if SQL statements were already executed during flushing.

Remember: Save manages the entity, flush sends pending SQL to the database, and commit makes the transaction permanent.

When multiple resource try to access a critical resource data inconcensectancy can happen @Trasaction can help us on this

Ex: booking flight ticket for praticular seat

It help us to achieve acid properties:

A(Atomicity): Ensures all operation in transaction are completed successfully ,if any operation failed entire transaction is rolled back

C(Consistency): ensure before and after transaction is consistent

I(Isolation): if multiple transaction are running on parrallel they do not interfare each other

D(Durability): it ensures even their is system failure transaction will not be lost.

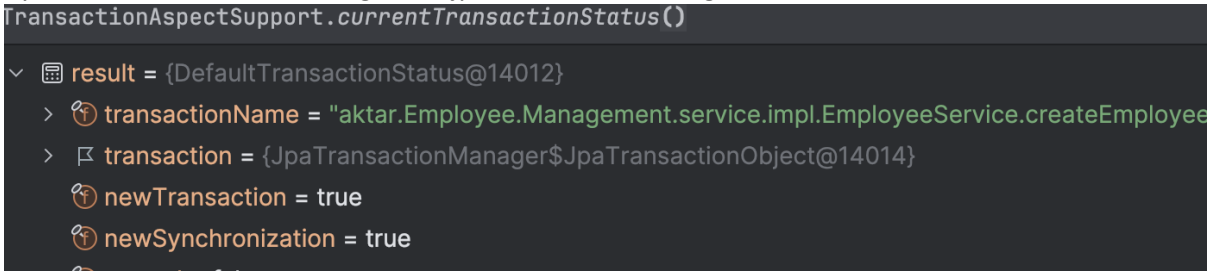
Spring data jpa needed in pom

@Trasactional : can be apply on class/method level

@Transactional **does NOT work on private methods**.



By default transactional manager use jpa transaction manager



1.Declarative

->transactional management through annotaion

2.programatic

->Handke transac programatically through the code.

Transaction **propagation** refers to how transaction behave when a method is annotated with @transa which is called by another which may or may not have annotated with trasaction.

@Transactional(propagation=Progation.REQUIRED) default one same as @Transactional

@Transactional(ropagation=Progation.REQUIRED_NEW) it will start new transaction ,some method we keep isolated in that case we can use , it will suspend the existing trasactyion from calling method .it will not join the existing transaction like default one.

MANDATORY: calling method also should have transaction otherwise it will throw except

ion

Not supported: for that perticular method trasaction will get skipped

Support: if calling method have trasaction then only it will happen

Never: calling method should not have any trascation, we are not performing trasaction for the particular method.

Why @Transactional is needed

When **multiple users/resources** try to access or modify the **same critical data** at the same time, **data inconsistency** can occur.

Example

✈️ **Flight seat booking**
Two users try to book **Seat A1** simultaneously.
Without transactions → **double booking** may happen.

👉 @Transactional helps **maintain data integrity**.

ACID Properties

@Transactional helps achieve **ACID** properties:

A – Atomicity

- All operations inside a transaction must succeed.
- If **any operation fails**, the **entire transaction is rolled back**.

Example:
Seat booked ❌ → Payment failed ❌ → Seat booking rolled back

C – Consistency

- Ensures the database remains in a **valid state** before and after the transaction.

Example:
Seat count before booking = 10
Seat count after booking = 9 ✓

I – Isolation

- Multiple transactions running **in parallel do not interfere** with each other.
- One transaction cannot see **partial changes** of another.

Example:
User A booking seat does not affect User B's transaction until commit.

D – Durability

- Once a transaction is **committed**, it is **permanently saved**.
- Even system crash or restart won't lose the data.

Spring & Transactions

Required Dependency

```
<dependency>
   <groupId>org.springframework.boot</groupId>
   <artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>
```

@Transactional Basics

- Can be applied at:
    - **Class level**
    - **Method level**
- Method-level annotation **overrides class-level**

```
@Transactional
public void bookTicket() { }
```

Important Rules

❌ @Transactional does NOT work on private methods

Reason:

- Spring uses **proxies**
- Private methods **cannot be intercepted**

Default Transaction Manager

- Spring Boot uses **JpaTransactionManager** by default (when using JPA)

Types of Transaction Management

1️⃣ Declarative Transaction Management (Most Common)

- Using annotations

```
@Transactional
public void saveEmployee() { }
```

✓ Recommended
✓ Cleaner
✓ Less error-prone

2️⃣ Programmatic Transaction Management

- Transaction handled manually in code

```
TransactionStatus status = transactionManager.getTransaction(definition);
```

❌ Complex
❌ Rarely used in real projects

Transaction Propagation

Transaction propagation defines how a method behaves when it is called from another transactional method.

1️⃣ REQUIRED (Default)

@Transactional(propagation = Propagation.REQUIRED)

- Joins existing transaction if present
- Otherwise creates a new one

✓ Same as @Transactional

2️⃣ REQUIRES_NEW

@Transactional(propagation = Propagation.REQUIRES_NEW)

- Always starts a **new transaction**
- Suspends existing transaction

📌 Used when logic must be **isolated**
Example: logging, audit records

3️⃣ MANDATORY

@Transactional(propagation = Propagation.MANDATORY)

- Calling method **must already have a transaction**
- Otherwise → **Exception**

4️⃣ NOT_SUPPORTED

@Transactional(propagation = Propagation.NOT_SUPPORTED)

- Transaction is **suspended**
- Method executes **without transaction**

5️⃣ SUPPORTS

@Transactional(propagation = Propagation.SUPPORTS)

- Uses transaction **only if caller has one**
- Otherwise runs without transaction

6️⃣ NEVER

@Transactional(propagation = Propagation.NEVER)

- Method **must not** run inside a transaction
- If transaction exists → **Exception**

Interview One-Line Summary 🔥

@Transactional ensures data consistency by managing transactions using ACID properties. It works through Spring proxies, supports declarative and programmatic transaction management, and provides multiple propagation behaviours to control transactional execution.

By using transactions, you can ensure that your database updates are executed in a consistent and reliable way, and that any errors are properly handled and rolled back.

Isolation defines how and when the changes made by one transaction become visible to other concurrent transactions.

Isolation	Problem
READ_UNCOMMITTED	Two users see same seat
READ_COMMITTED	Seat may change after read
REPEATABLE_READ	Seat stable, new seats may appear
SERIALIZABLE	Only one booking at a time

@Transactional(isolation = Isolation.READ_COMMITTED)

Spring Boot Actuator is used for the production applications. If you are deploying your application to production, then it is important to periodically check the health of your application, metrics, log files, etc. without disturbing the application environment. These services can be exposed as the endpoints when you start your application.

Spring data jpa

ORM: Object relation mapping it mappes our object to db entity,but update ,rertrieve for that we need framework ex: hibernate

ORM is a technique that allows developers to interact with the database using Java objects instead of SQL queries.

Jpa: **JPA is NOT a framework** ❌
JPA is a specification (rules) ✅

It defines:

- How ORM should work
- Common annotations & interfaces

Jpa annotation

@Entity

@Id

@Table

Hibernate implements these annotations internally.

Hibernate is:

- A **Java ORM framework**
- Implements ORM concept
- Handles database operations automatically

What Hibernate does?

- Converts Java class → DB table
- Converts object → row
- Generates SQL internally
- Handles transactions & caching

JPA	Hibernate
Specification	Implementation
Defines rules	Implements rules
	Actual code
Interfaces	

JPA is a specification that defines ORM standards, while Hibernate is an implementation of JPA.

User user = new User("Nasima");
userRepository.save(user);
What happens?

Spring Data JPA calls JPA

JPA uses Hibernate

Hibernate generates SQL

SQL runs on DB

Spring Data JPA simplifies database access by providing ready-made CRUD methods on top of JPA and Hibernate. if we use this extendds jpa repo

```
public interface UserRepository
    extends JpaRepository<User, Long> {
}
```

You automatically get:

- save(),findById(),findAll(),delete()

Persistance context: temporary store of entity.

☑ Hibernate

✓ A **Java framework**
✓ Used for **ORM (Object Relational Mapping)**
✓ Helps map Java objects ↔ Database tables
✓ Works with databases like MySQL, PostgreSQL, Oracle, etc.

Example:
Mapping a Java class User to a DB table users.

Mongodb->spring data mongodb extension needed no need of jpa cause it is a NO sql db not a realtional db

MongoRepository : is a interface which is provided by spring data mongodb,it does standard crud op.

Lombok : it reduces the boiler plate of code no need to getter,setter. It will write automatically during compilation time

Scheduling

It allows to automate a task reduce manual effort

@EnableScheduling // Enable Spring's scheduled task execution in main method

@Scheduled(fixed rate=5000): it may overlaped

Fixed delay: wait for 5 sec after each time meyhod got called

Initial delay: before callleing method

```
@Scheduled(cron = "0 * * * * *") // Cron expression for running every minute
 public void execute() {
 demoService.dummyMethod();
 }
```

Dynamic scheduling allows tasks to be scheduled, modified, or cancelled at runtime using TaskScheduler instead of fixed @Scheduled annotations.

FixedRate = Every 5 sec no matter what
👉 **FixedDelay = Finish → Wait 5 sec → Start**

🎯 Interview One-Line Answer

fixedRate schedules based on start time of previous execution, while fixedDelay schedules after the previous execution completes.

@ConditionalOnProperty:

Toggling of feature i mean migrating to differnt feature,while our application is coming up we can avoid some beans by using this annotation, it save s our memory also, reduce the application startup time

How @ConditionalOnProperty Works?

```
@ConditionalOnProperty(prefix = "phonepay",value="enabled",havingValue = "cre ate",matchIfMissing = false)
```

If the specified property exists and matches havingValue, the bean is created.

If the property does not exist and matchIfMissing = false, the bean is not created.

If matchIfMissing = true, the bean is always created, even if the property is missing.

When to Use @ConditionalOnProperty?

Feature Toggle — Enable or disable features dynamically using properties.

Multiple Environment Support — Activate different configurations per environment (e.g., dev, prod).

Enable/Disable Logging, Security, or Background Jobs dynamically.

How to set profile?

1.In application startup we can do that using the below cmd , if the dev is not present at that time it will pick up from parent property , evn the file is present but no content it check in parent again but it will not cause any issue.

In main application.prop file need to mention Spring.active.profiles=dev

2.Dynamic setting profile while running the code no need of code change

mvn spring-boot:run -Dspring-boot.run.profiles=qa

3.This is another way adding in pom

```
<profiles>
    <profile>
        <id>local</id>
        <properties>
        <spring-boot.run.profiles>dev</spring-boot.run.profiles>
        </properties>
    </profile>
```

mvn spring-boot:run -Pdev

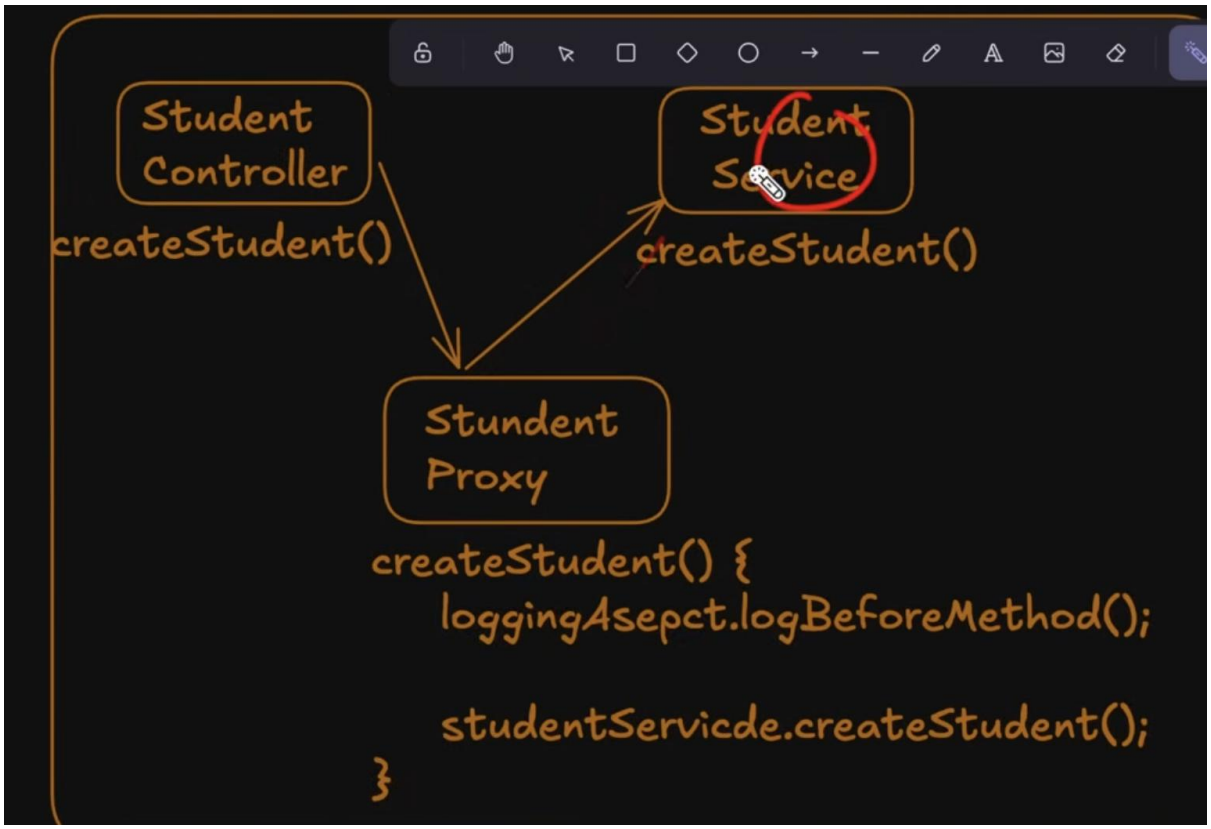
```java
@Profile("dev")
public class PhonePayService {
    @Value("${username}") 1 usage
    String userName;
    @Value("${password}") 1 usage
    String password;
```

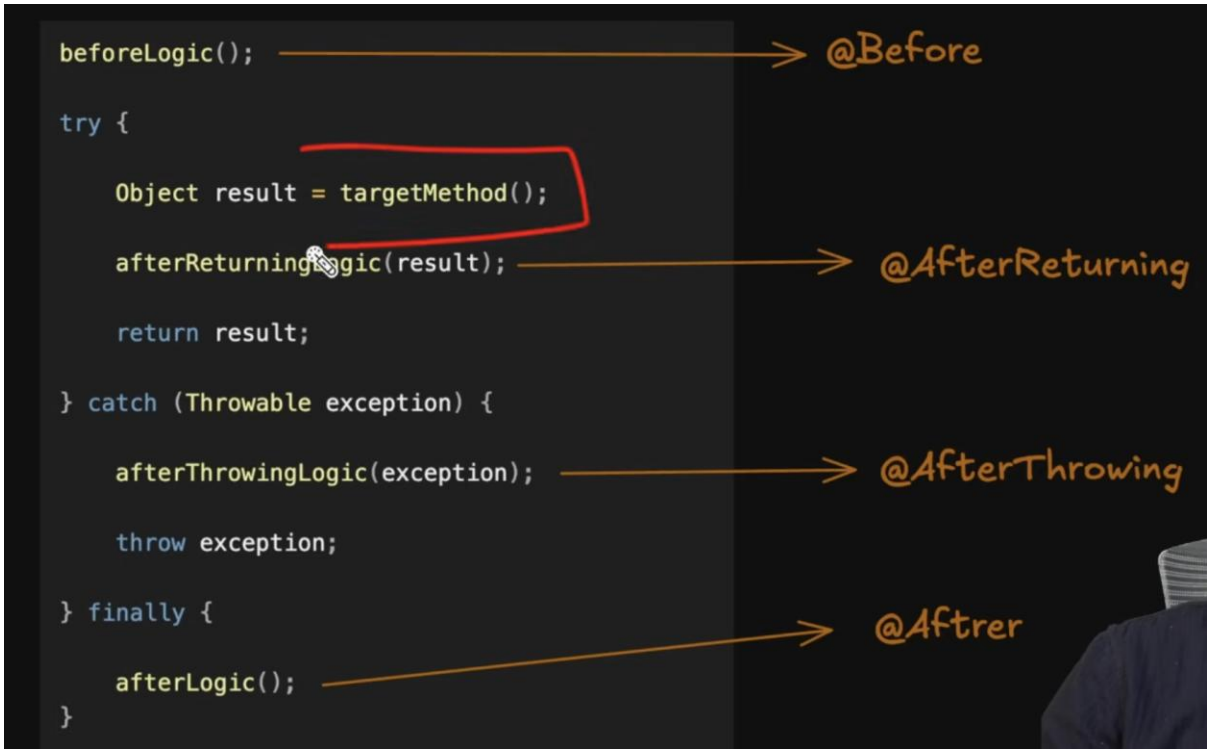
@Profile helps spring to identify the beans that belong to a particular environment.(dev/qa/prod), as the upper example bean will get initialized in dev only.

@Profile is applied at class level except for the classes annotated

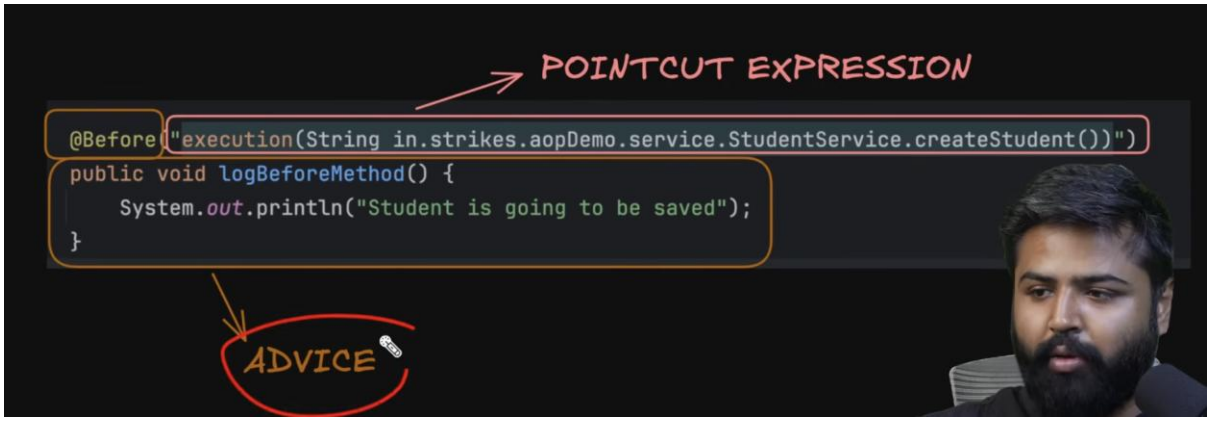
with @Configuration where @Profile is applied at the method level.

Aspect oriented programming




beforeLogic();
@Before
try {
Object result = targetMethod();
afterReturningLogic(result);
@AfterReturning
return result;
} catch (Throwable exception) {
afterThrowingLogic(exception);
@AfterThrowing
throw exception;
} finally {
@Aftrer
afterLogic();
}


@Pointcut("within(in.strikes.aopDemo.service..*) " +
"&&" +
"execution(public * * (..))")
public void logPublicServiceMethod() {
// empty body
}
@Before("logPublicServiceMethod()")
public void logBeforeMethod() {
System.out.println("Method Intercepted");
}


POINTCUT EXPRESSION
@Before("execution(String in.strikes.aopDemo.service.StudentService.createStudent())")
public void logBeforeMethod() {
System.out.println("Student is going to be saved");
}
ADVICE

@Around

It can controlls out service method , it can change the argument and even it can change the return time

@Around advice can control the execution of the target method. It can modify the method arguments before the method executes, decide whether to execute the method, and modify the return value after the method executes.

If you don't call joinPoint.proceed(), the actual service method will not execute.

That's why @Around is more powerful than @Before and @After.

@EnableAspectJAutoProxy(proxyTargetClass=true)

Spring @EnableAspectJAutoProxy annotation used with @Configuration annotation enables support for handling components marked with @Aspect annotation of AspectJ.

logger.info("Around AFTER execution");

@ are for basically wherever @service/@component or any annotation will be there it be apply on that

And like within will be for classes

Intercept means catching a method call before it executes, to add extra behavior like logging, transactions, or security, and then letting the method continue.

A proxy is a wrapper object created by Spring to intercept method calls and apply cross-cutting concerns like transactions, logging, security, and caching before and after executing the actual method.

Join Point 🎯
A point in your program where an action can be taken, like a method invocation.

It ‘s like security guard

@Around: it wil get callled before and after executing the method

@Before:before that method

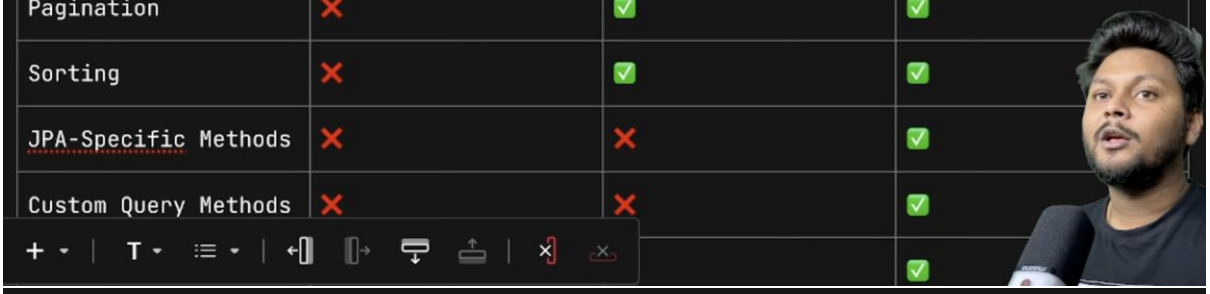
Advise: this type are called(before,after etc)

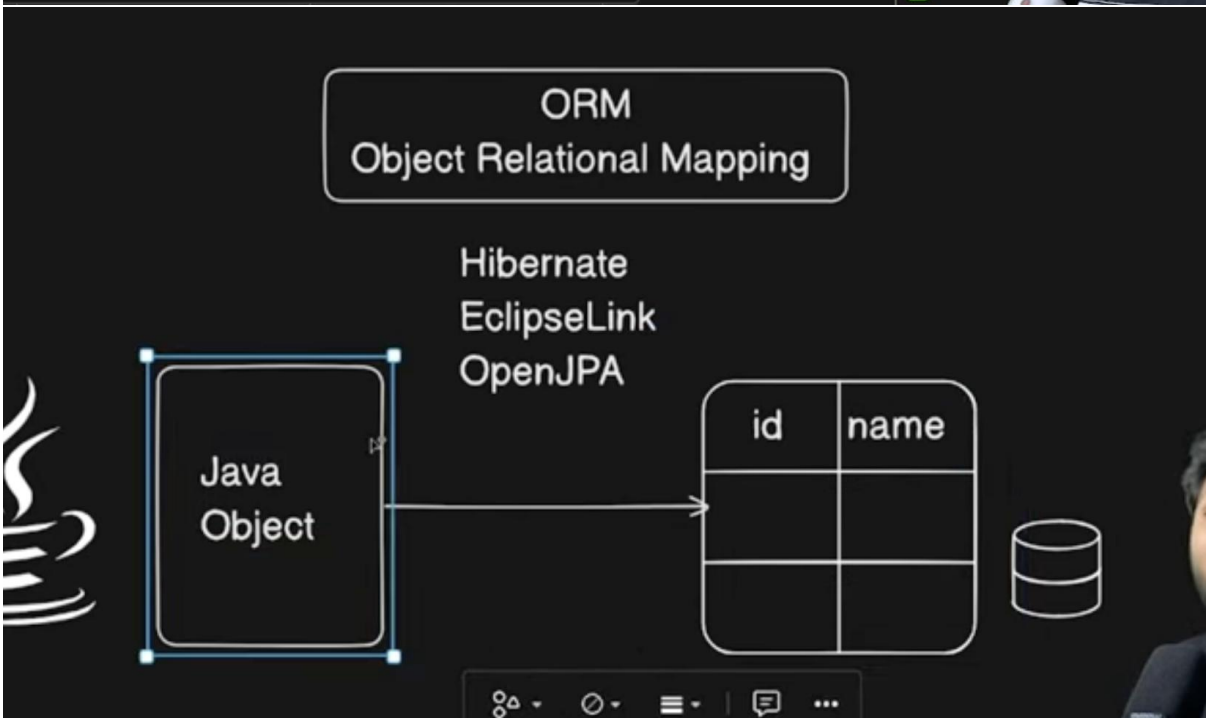


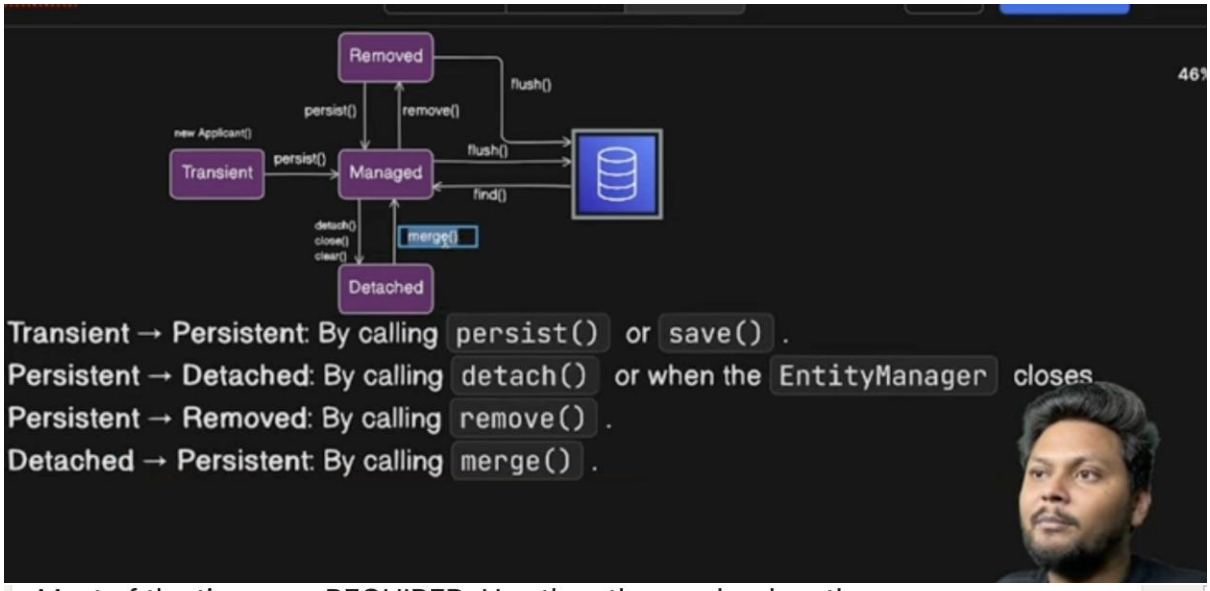
```
@Component  no usages
@Aspect
public class LoggingAspect {
    @Pointcut("within(com.codesnippet.ecom.Service.ProductService) && @within(org.springframework.stereotype.Service)")
    private void anyOldTransfer() {}

    @Around("anyOldTransfer()")  no usages
    public Object log(ProceedingJoinPoint jointPoint) throws Throwable {
        System.out.println("Aspect Before log called");
        Object val= jointPoint.proceed();
        System.out.println("Aspect After log called");
        return val;
    }
}
```

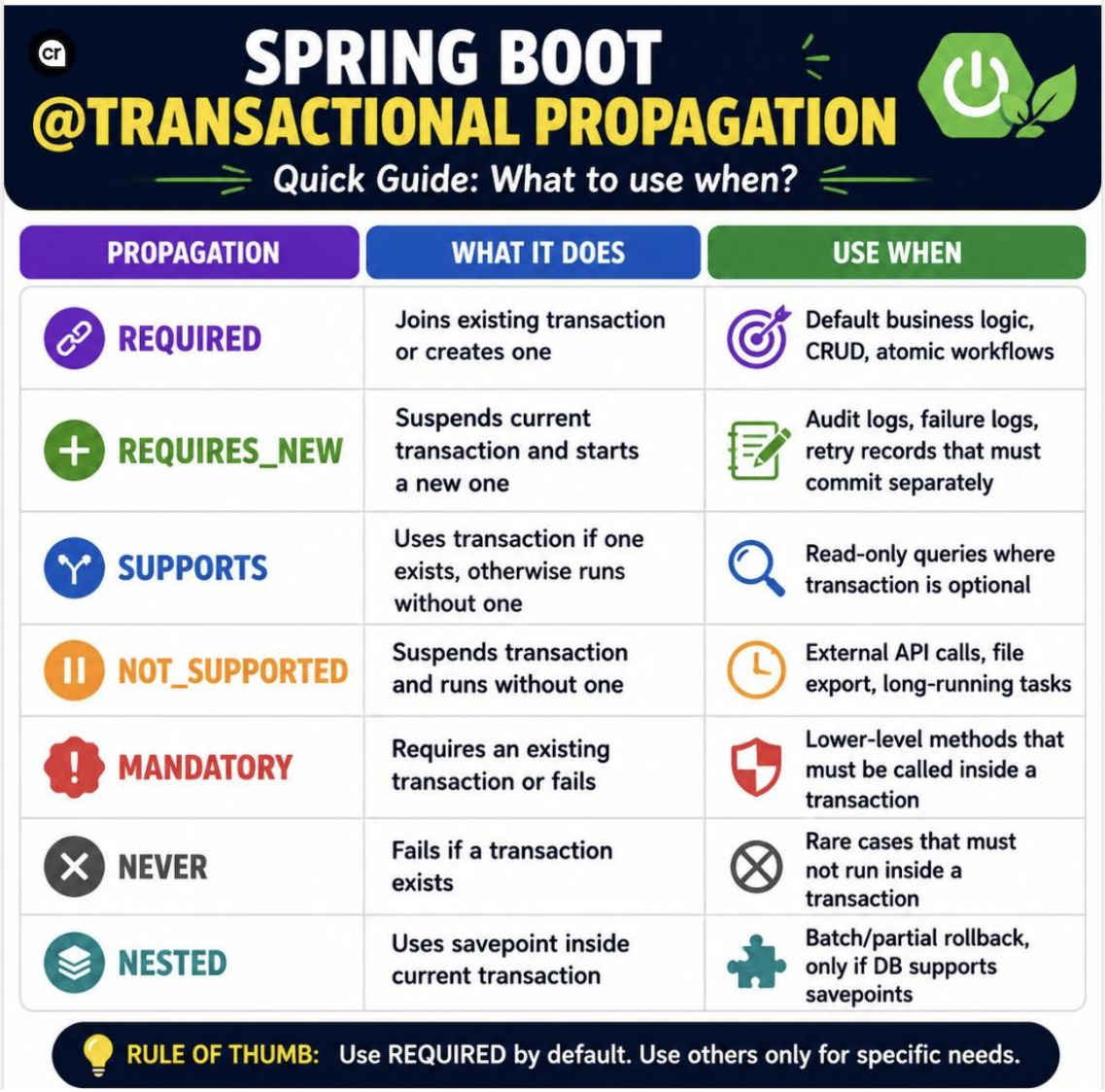
Feature	CrudRepository	PagingAndSortingRepository	JpaRepository
Basic CRUD Operations	✅	✅	✅
Pagination	❌	✅	✅
Sorting	❌	✅	✅
JPA-Specific Methods	❌	❌	✅
Custom Query Methods	❌	❌	✅
			✅







Most of the time, use REQUIRED. Use the others only when the... more



```
public interface ApplicantJpaRepository extends JpaRepository<Applicant,Long> {  2 usages
    Rename usages
    List<Applicant> findByStatusOrderByNameAsc(String status);

}
```

@Query: help use to create query which jpa can not provide

@joinColumn(name="")= for foreign key

@OneToOne

Why use mappedBy?

Without mappedBy, JPA creates an extra join table unnecessarily.

mappedBy tells:

relationship already managed by Employee entity.

```
    private String status;  2 usages

    @OneToOne(mappedBy = "applicant",cascade = CascadeType.ALL)  2 usages
    private Resume resume;

    @OneToMany(mappedBy = "applicant", cascade = CascadeType.ALL)  2 usages
    private List<Application> applications = new ArrayList<>();

    @ManyToMany  no usages
    @JoinTable
    private List<Job> jobs = new ArrayList<>();
```


```
    private String status;  2 usages

    @OneToOne(mappedBy = "applicant",cascade = CascadeType.ALL)  2 usages
    private Resume resume;

    @OneToMany(mappedBy = "applicant", cascade = CascadeType.ALL)  2 usages
    private List<Application> applications = new ArrayList<>();

    @ManyToMany  no usages
    @JoinTable
    private List<Job> jobs = new ArrayList<>();
```

Spring Security

It helps to protect api,web app, microservices from unathoraized access

Authentication: it verifies who you are are

Autheraization: it basically meanns what you can acces, what you can do inside your application

Spring boot starter security need to ad in pom

JWT is a token-based authentication mechanism used to secure APIs. In our Spring Boot project, after user login, JWT Bearer token was generated and sent from UI in the Authorization header for every API call. Spring Security validated the token before allowing access to APIs. We used Spring Security filters, role-based authorization, and short-lived access tokens for better security.

What is JWT Security?

JWT stands for:

JSON Web Token

It is used for:

- authentication
- authorization
- securing REST APIs

JWT is a compact token sent between client and server.

JWT Structure

JWT has 3 parts:

Header.Payload.Signature

User Login

↓

Username/password validated

↓

JWT token generated

↓

UI stores token

↓

Token sent in every API request

↓

Backend validates token

↓

Access granted

How I Used JWT in My Project

You can say:

In our SMO Spring Boot application, we used JWT Bearer token authentication to secure backend APIs. After successful login, the authentication service generated a JWT access token. The UI sent this token in the Authorization header before calling APIs. Backend validated the token expiry and signature using Spring Security filters before allowing access to secured endpoints.

Java cannot sort the custom object the way it can do for primitive obj that is why comparable and comparator is needed.

Kafka

Produces: send message to kafka,publishes message to a topic, doesnt know which cousumer read the data, it can choose pertition using keys

Topic: stream of data ,messages are stored inside the partition. A topic is like a folder where messages are stored.

Partioton : it is a physical storage unit of kafka topic,messages inside a partiotion is ordered

Consumer: read ,essage from toppic

Cosumer group: group of the cosumer are working together.each partion can get connected with only one consumer.

Broker:kafka server that stores the data,handles producer and consumer request

Cluster =multiple broker

Offset: every partion get a unique number in partiotion,kafka tracks the which offset the cosumer have already processed.

Key: it decides which partion get the meaages,it not mandatory

Microservice

Monolithic: as a single unit

Dis advantage: Re deployment full application,lots of dependency

Microservice:

Code base are independent

Independent deployment

Flexibility scaling

Dis Advantage:cost is much

Synchronous communication

Through api calls services are communicate to each other(restapi/http)

Asynchronous communication

It uses message broker ex:Kafka

◇ What is Microservices?

Microservices architecture is a design where an application is divided into **small, independent services**, each responsible for a **single business functionality**.

✓ Independent development
✓ Independent deployment
✓ Separate codebase
✓ Can use different technologies

👉 Each service is **loosely coupled and highly cohesive**

◇ API Gateway

Acts as the **entry point** for all client requests.

Responsibilities:

- Routes requests to appropriate service
- Authentication & authorization
- Load balancing
- Rate limiting
- Logging

👉 Example: Spring Cloud Gateway

◇ Service Discovery

Keeps track of all running service instances and their locations.

- Helps gateway/services find each other dynamically
- Routes request to **healthy instance**

👉 Example: Netflix Eureka

◇ Service Registry

- Database of available service instances
- Stores IP + port of each service

👉 Eureka acts as both **service discovery + registry**

◇ Heartbeat

- Services send periodic heartbeats to registry
- If heartbeat fails → service is removed (marked unhealthy)

◇ Circuit Breaker

Cascading failure: when one component failing is causing in dependent componet failure

Prevents repeated calls to a failing service.

Circuit Breaker is a protective and safety mechanism that prevents your application from continuously making requests to a service that has problems or is down.

States:

- Closed → Normal flow
- Open → Calls blocked
- Half-open → Trial requests

👉 Example: Resilience4j

◇ CORS (Cross-Origin Resource Sharing)

- Controls which frontend domains can access backend APIs
- Used for **security in browser-based applications**

◇ CQRS (Command Query Responsibility Segregation)

Command Query Responsibility Segregation (CQRS) is an architectural pattern that improves scalability and performance by separating read and write operations into distinct models. Instead of using a single model to both retrieve and modify data, CQRS divides responsibilities into two parts:

- Commands, which handle state-changing operations such as insert, update, and delete.
- Queries, which retrieve data without modifying the system state.

The **Command Query Responsibility Segregation (CQRS)** design pattern separates the operations for reading data from the operations for writing or updating data

- Separates:
  - Command → Write operations
  - Query → Read operations

👉 Improves performance and scalability

Saga patter1n

The **Saga design pattern** is a failure management strategy for maintaining data consistency in a microservices architecture without relying on traditional distributed transactions.

1. **Choreography (Decentralized):**
   a. **How it works:** Participants exchange events without a central controller. Each service listens for events from others and acts accordingly.
   b. **Pros:** Simple for small workflows, loose coupling, and no single point of failure.
   c. **Cons:** Hard to track the workflow as steps increase; risk of cyclic dependencies between services.
2. **Orchestration (Centralized):**

a. **How it works:** A central **Saga Orchestrator** (a dedicated object or service) tells participants what local transactions to execute and manages the entire workflow.

soli

◇ Strangler Pattern

- Gradually replaces a monolithic application with microservices
- Old and new systems run together during migration

🔥 Important Additional Concepts (Must Know)

◇ Inter-Service Communication

How services communicate:

- REST API → Synchronous
- Messaging → Asynchronous

👉 Example: Apache Kafka

◇ Database per Service

- Each microservice should have its own database

✓ Ensures loose coupling
✓ Independent scaling

◇ Load Balancing

Distributes traffic across multiple service instances.

◇ Fault Tolerance

Ensures system stability using:

- Circuit Breaker
- Retry
- Timeout
- Fallback

◇ Configuration Management

Centralized configuration for all services.

👉 Example: Spring Cloud Config

◇ Containerization & Deployment

- Services are packaged using Docker
- Deployed and managed using Kubernetes

🎯 One-Line Interview Summary

Microservices architecture divides an application into small, independent services that communicate via APIs, use separate databases, and are deployed independently using tools like Docker

and Kubernetes, with support from components like API Gateway, service discovery, and circuit breakers.

Synchronus communication:

Everything is step by step . One service calls another service and waits for response.

Design pattern



Single tone : for a class obj creation is one and use it every where

Factory patter: hidden the obj creation

m princples:

1. S — Single Responsibility Principle

Real Application

In **Spring Boot**

You usually see:

UserController
UserService
UserRepository

Each has **separate responsibility**.

2.O — Open Closed Principle (OCP)

S → Single job

O → Extend without modifying

L → Child must behave like parent

I → Small interfaces

D → Depend on abstraction

Design Patterns & SOLID – Clean Notes

◇ 1. Singleton Pattern

✔ Definition

Ensures that a class has **only one instance** and provides a **global access point** to it.

✔ Why use?

- Shared resources (config, logger, cache)

✔ Example (Real Life)

- Database connection manager
- Logging service

✔ In Spring Boot

👉 Beans are **Singleton by default**

🎯 One-Liner:

Singleton ensures only one instance of a class exists and is used throughout the application.

◇ 2. Factory Pattern

✔ Definition

Provides an interface to create objects, but **hides the object creation logic**.

✔ Why use?

- Avoid tight coupling
- Makes code flexible

✔ Example

Instead of:

```
new Dog();
new Cat();
```

Use:

Animal a = AnimalFactory.getAnimal("dog");

✔ In Spring Boot

- Spring internally uses Factory Pattern to create beans

🎯 One-Liner:

Factory pattern hides object creation logic and provides objects through a common interface.

🔥 3. SOLID Principles (Very Important)

◇ S — Single Responsibility Principle (SRP)

✔ Definition:

A class should have **only one reason to change**.

✔ Your Example (Correct 👍):

- Controller → handles API
- Service → business logic
- Repository → DB

🎯 One-Liner:

A class should do only one job.

◇ O — Open/Closed Principle (OCP)

✔ Definition:

- Open for extension
- Closed for modification

✔ Meaning:

👉 Add new functionality **without changing existing code**

🎯 One-Liner:

Code should be extendable without modifying existing code.

◇ L — Liskov Substitution Principle (LSP)

✔ Definition:

Child class should **replace parent class without breaking behavior**

✔ Example:

If Bird has fly()
👉 Penguin should NOT extend Bird (because it can't fly)

🎯 One-Liner:

Subclasses should behave correctly when used as their parent.

◇ I — Interface Segregation Principle (ISP)

✔ Definition:

Don’t force a class to implement methods it doesn’t need.

✔ Example:

❌ Bad:

```
interface Animal {
  eat();
  fly();
}
```

👉 Dog forced to implement fly ❌

✔ Good:

```
interface Eatable { eat(); }
interface Flyable { fly(); }
```

🎯 One-Liner:

Use small, specific interfaces instead of large ones.

◇ D — Dependency Inversion Principle (DIP)

✔ Definition:

Depend on **abstractions (interfaces)**, not concrete classes.

✔ Example:

@Autowired
PaymentService paymentService;

👉 Not:

StripePaymentService paymentService;

✔ In Spring Boot

- Achieved using **Dependency Injection**

🎯 One-Liner:

High-level modules should depend on abstractions, not implementations.

🎯 Final Short Answer (Interview Ready)

Singleton pattern ensures only one instance of a class is created. Factory pattern hides object creation logic.

SOLID principles are five design principles:
SRP ensures a class has a single responsibility, OCP allows extension without modifying code, LSP ensures child classes behave like parent, ISP promotes small interfaces, and DIP encourages dependency on abstractions using dependency injection.

```java
import java.util.Scanner;
public class Function{
    public static int printProduct(int a,int b){
        int product=a*b;
        return product;
    }
    public static void main(String args[]){
        Scanner sc=new Scanner(System.in);
        int a=sc.nextInt();
        int b=sc.nextInt();
        int prod=printProduct(a,b);
        System.out.println(prod);


    }
}
```

Fact of number

```java
import java.util.Scanner;
public class Function{
    public static int printProduct(int n){
        int fact=1;
        for(int i=1;i<=n;i++){
            fact=fact*i;
        }
        return fact;
    }
    public static void main(String args[]){
        Scanner sc=new Scanner(System.in);
        int a=sc.nextInt();
        int factor=printProduct(a);
        System.out.println(factor);


    }
}
```

If we can a function use of object then it called method but if call directly it will be function only

Sum of odd number 1 to n.

```java
import java.util.Scanner;
public class Function{
    public static void printProduct(int n){
        int sum=0;
        for(int i=1;i<=n;i++){
            if(i%2!=0){
                sum+=i;
            }
        }
        System.out.println("Sum of odd numbers from 1 to " + n + " is: " + sum);
    }
    public static void main(String args[]){
        Scanner sc=new Scanner(System.in);
        int a=sc.nextInt();


        printProduct(a);
    }
}
```

Store marks in array and print

```java
import java.util.Scanner;
public class Function{

    public static void main(String args[]){
        int marks[]={6,3,4,9};
        int size=marks.length;
        for(int i=0;i<size;i++){
            System.out.println(marks[i]);
        }
    }
}
```

Searching

```
    public static void main(String args[]){
        Scanner sc=new Scanner(System.in);
        int size=sc.nextInt();
        int marks[]=new int[size];
        for(int i=0;i<size;i++){
            marks[i]=sc.nextInt();
        }
    System.out.println("enter to search: ");
        int elem=sc.nextInt();
        int found=0;
        for(int i=0;i<size;i++){
            if(marks[i]==elem){
                found=1;
            }

        }

        if(found>=1){
            System.out.println("present");
        }
        else{
           System.out.println(" not present");
        }
      }
```

```
public static void main(String args[]){

    Scanner sc=new Scanner(System.in);
    int size=sc.nextInt();
    int marks[]=new int[size];
    for(int i=0;i<size;i++){
        marks[i]=sc.nextInt();
    }
System.out.println("enter to search: ");
    int elem=sc.nextInt();
    int found=0;
    for(int i=0;i<size;i++){
        if(marks[i]==elem){
            found=1;
            System.out.println("found at index:"+i);
        }


    }
    if(found==0){
       System.out.println("no");
        }
    }
```

Palindrome

```java
import java.util.Scanner;
public class Function{

    public static void main(String args[]){

        Scanner sc=new Scanner(System.in);
        int number=sc.nextInt();
        int temp=number;
        int result=0;
        while(number>0){
            int remainder=number%10;
            result=result*10+remainder;
            number=number/10;
        }
        if(temp==result){
            System.out.println("palindrome");
        }
        else{
            System.out.println("not palindrome");
        }


        }

```

Print string char

```java
import java.util.Scanner;
public class Function{

    public static void main(String args[]){

        Scanner sc=new Scanner(System.in);
       String name="kuhu aktar";
       System.out.println(name.length());
       for(int i=0;i<name.length();i++){
           System.out.println(name.charAt(i));
       }

        }


}
```

Compare string

```java
import java.util.Scanner;
public class Function{

    public static void main(String args[]){

        Scanner sc=new Scanner(System.in);
       String name="nasima";
       String name2="nasima aktar";
       if(name.compareTo(name2)==0){
           System.out.println("equal");
       }
       else{
           System.out.println("not equal");
       }



        }
```

Substring(starting point,ending point)

```java
import java.util.Scanner;
public class Function{

    public static void main(String args[]){

        Scanner sc=new Scanner(System.in);
       String name="nasima altar";
       String sub=name.substring(0,5);

       System.out.println(sub);


        }



}
```

Print array of string and count the length of full array

```java
import java.util.Scanner;
public class Function{

    public static void main(String args[]){

        Scanner sc=new Scanner(System.in);
        System.out.println("enter number of strings");
        int size=sc.nextInt();
        sc.nextLine();
       String name[]=new String[size];
      int length=0;
      System.out.println("enter sirings: ");
      for(int i=0;i<size;i++){
          name[i]=sc.nextLine();
          length=length+name[i].length();
      }
       System.out.println(length);

        }


```

Relace e with I and store in a new variable

```java
public class Function{

    public static void main(String args[]){

        Scanner sc=new Scanner(System.in);
        System.out.println("enter name");
        String name=sc.nextLine();
        int size=name.length();
        String result="";
        for(int i=0;i<size;i++){
            if(name.charAt(i)=='e'){
                result=result+'i';

            }
            else{
                result=result+name.charAt(i);
            }
        }
        System.out.println(result);

    }
```

take email then remove after @ then store in a new var and print

```java
public class Function{

    public static void main(String args[]){

        Scanner sc=new Scanner(System.in);
        // System.out.println("enter name");
        String name="nasimaaktar@98";
        String name2="";
       int size=name.length();
       for(int i=0;i<size;i++){
           if(name.charAt(i)=='@'){
               break;
           }
           else{
               name2=name2+name.charAt(i);

           }
       }
       System.out.println(name2);
    }
```

Conversion

```java
import java.util.Scanner;
public class Function{

    public static void main(String args[]){

        Scanner sc=new Scanner(System.in);
        // System.out.println("enter name");
        int number=123;
        //convert to string

        String str=Integer.toString(number);
        System.out.println(str);
        String name="123";
        //convert to int
        int age=Integer.parseInt(name);
        System.out.println(age);
    }
```

Copy one string to another string one by one char

name2=name2+name.charAt(i);

String buffer operation

```java
import java.util.Scanner;
public class Function{

    public static void main(String args[]){
        String name="ssima";
       StringBuilder sb=new StringBuilder(name);
       System.out.println(sb);
       //change a char at a particular position
        sb.setCharAt(0,'a');
        System.out.println(sb);
        //inserting new char
        sb.insert(0,'n');
        System.out.println(sb);
        //delete char
        sb.delete(1,2);
        System.out.println(sb);
        sb.append("aktar");
        System.out.println(sb);

    }
```

Reverse String

```java
import java.util.Scanner;
public class Function{

    public static void main(String args[]){
        String name="nasima";
        int size=name.length();
        char newName[]=name.toCharArray();
        int start=0;
        int end=size-1;
        while(start<end){

            char temp=newName[start];
            newName[start]=newName[end];
            newName[end]=temp;
            start++;
            end--;
        }
        String reverseString=new String(newName);

       System.out.println(reverseString);

    }
```

```java
import java.util.Scanner;
public class Function{

    public static void main(String args[]){
        String name="nasima";
        StringBuilder sb=new StringBuilder(name);
        int size=name.length();
        for(int i=0;i<size;i++){
            int front =0;
            int end=size-i-1;
            char frontchar=sb.charAt(front);
            char endchar=sb.charAt(end);
            sb.setCharAt(front,endchar);
            sb.setCharAt(end,frontchar);

        }
        System.out.println(sb);

    }
```

Palindrome string or not

```java
    public static void main(String args[]){
        String name="madam";
        int size=name.length();
        char charName[]=name.toCharArray();
        int start=0;
        int end=size-1;
        while(start<end){
            char temp=charName[start];
            charName[start]=charName[end];
            charName[end]=temp;
            start++;
            end--;
        }
        String reverse=new String(charName);
        System.out.println(reverse);
        System.out.println(name);
        if(name.equals(reverse)){
            System.out.println("palindrom");
        }
        else{
            System.out.println("not palindrome");
        }
```

```java
import java.util.Scanner;
//import java.util.Arrays;
class HelloWorld {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        String name="miam";
        int size=name.length();
        String reverse="";
        for(int i=size-1;i>=0;i--){
            reverse=reverse+name.charAt(i);
        }
        System.out.println(reverse);
       if(name.equals(reverse)){
           System.out.println("yes pal");
       }
       else{
           System.out.println("not");
       }
    }
}
 //System.out.println("not");
```

```java
class HelloWorld {
    public static void main(String[] args) {
        String name=new String("kuhu");
        System.out.println(name);
        int size=name.length();
        boolean result=true;
        for(int i=0;i<=size/2;i++){
            if(name.charAt(i)!=name.charAt(size-i-1)){
                result=false;
                break;
            }
        }
        if(result){
            System.out.println("yes");

        }
        else{
            System.out.println("not");
        }
```

Bit masking

Main.java

```java
import java.util.Scanner;
public class Function{

    public static void main(String args[]){
        int n=5;
        int pos=2;
        int bitmask=1<<pos;
        if((bitmask &n )==0){
            System.out.println("switch off");
        }
        else{
             System.out.println("switch on");
        }

    }



}
```

Sort name according to length

```java

public class Practice {
    public static void sort(String name[],int heights[]) {
        for(int i=0;i<name.length;i++) {
            for(int j=i+1;j<name.length;j++) {
                if(heights[i]>heights[j]) {
                    int temp=heights[i];
                    heights[i]=heights[j];
                    heights[j]=temp;
                    String tempName=name[i];
                    name[i]=name[j];
                    name[j]=tempName;
                }
            }

        }
    }
    public static void main(String[] args) {
        String name[]={"Mary","John","Emma"};
        int heights[] = {180,165,170};

        Practice.sort(name,heights);
        for(int i=0;i<name.length;i++) {
            System.out.println(name[i]+""+heights[i]);
        }



    }
    }
```

Check if a linkedlist is palindrome
Eg. head: 1->2->1->1->2->1 output: True
head: 1->2->3->4 output: False

2. Given a string S containing only 0's and 1's, and given an integer N > 0, find the number of ways the string S can be split into N parts, with each part having at least 1 character, such that each part will have an equal number of 1's in it.

Eg. S = 101, N = 2
Answer: 2
Explanation: 2 possible ways to divide: "1" and "01", "10" and "1"
Q2: Nth Fibonacci number.
Input: N = 100
Solution: Use memoization or space optimization to handle large inputs.
Output: The 100th Fibonacci number.
Q3: Find the second largest element in an array.
Input: [12, 5, 8, 19, 29]
Output: 19 (second largest).

2. Technical Round 1:
Q1: Check if two strings are anagrams (input strings may contain spaces and a mix of upper and lower cases).
Input: "Listen", "Silent"
Output: True (they are anagrams).
Q2: Find the longest common prefix in a list of strings.
Input: ["flower", "flow", "flight"]
Output: "fl"
Q3: Factorial of a number using recursion.
Input: 5
Output: 120 (5! = 5 * 4 * 3 * 2 * 1).

3. Technical Round 2:
Q1: Simple question on a while loop with two variables count and ans.

```
int count = 0;
int ans = 1;
while(ans <= 2000) {
 count++;
 ans = ans * 7;
}
```

Output: Value of count when the loop exits: 5 (ans = 2401).
Q2: Calculate the salary of an employee based on a formula, where the employee

belongs to different teams (team1, team2, team3).

1. Given an unsorted array of integers, find the third largest number in the array. If the array has less than three distinct numbers, return the largest number.
Example:
Input:
arr = [2, 4, 1, 5, 3, 6]
Output:
4
Explanation:
The sorted array in descending order is [6, 5, 4, 3, 2, 1]. The third largest number is 4.

2. Given an array of strings, group the strings that are anagrams of each other.
Example:
Input:
arr = ["eat", "tea", "tan", "ate", "nat", "bat"]
Output:
[["eat", "tea", "ate"], ["tan", "nat"], ["bat"]]
Explanation:
The groups of anagrams are ["eat", "tea", "ate"], ["tan", "nat"], and ["bat"].

3. Given an array of integers, find the largest number that does not have any duplicates in the array.
Example:
Input:
arr = [4, 3, 2, 7, 3, 4, 8]
Output:
8
Explanation:
The unique numbers are [2, 7, 8], and the largest among them is 8.
Calculate Addition of Numbers in a String
Input: "23sdsdf 25sfs 24 wdsf"
Output: 72
1. Character Count of a String
2. Word Count of a String
3. Reverse a String
4. Reverse Only Words of a String
5. Add Student Record – Record with name and age; return sorted records by age.
6. Palindrome
7. Prime Number
8. 9. Print Different Types of Pyramid Patterns (Number or '*'), can be any pattern
Example:
1
23

456

*
**
***
10. Sort Numbers
11. Return Sorted Array
12. Return Maximum and Minimum Number from Array
13. Convert String to Array
14. Return Number of Word Occurrences in a String
15. Check Expression Validity
Example: "()(){{}{}}" – Should return true. Use stack for expression evaluation.
10)how to reverse the words in the string?eg: i like program language java to java language program like i
11)Using hash map,how to find no of occurences of a character in the string?
Question no. 1
Print prime number between 1-30

Question no. 2
Top K Frequent Elements Given a non-empty array of integers, return the k most frequent elements. Input: nums = [1,1,1,2,2,3], k = 2; Output: [1, 2]

Question no. 3
Subarray Sum Equals K Given an array of integers and an integer k, you need to find the total number of continuous subarrays whose sum equals to k. Input: nums = [1, 1, 1], k = 2; Output: 2

Question no. 4
Binary Tree In order Traversal Given the root of a binary tree, return its in order traversal. Input: [1,null,2,3]; Output: [1,3,2]

1) Why does a static block execute before the main method?
   ➡️ A class has to be loaded in main memory before we start using it. A static block is executed during class loading.

   2) Can we override the static method?
   ➡️ No, a static method cannot be overridden.

   3) Why can we not override the static method?
   ➡️ Static methods in Java cannot be overridden because a static method is bound to the class, whereas an instance method is bound to the object. Static belongs to the class area, and the instance belongs to the heap area.

   4) Why do we use the this keyword?

➡️ The `this` keyword is used to eliminate confusion between class attributes and parameters with the same name.

5) Difference between static and final keyword?
➡️ The `static` keyword always fixes the memory, meaning it will be located only once in the program, whereas the `final` keyword always fixes the value, making variable values constant.

6) Why is the main method static?
➡️ Because an object is not required to call a static method. If `main()` is a non-static method, then the JVM creates an object first and then calls `main()`, which can lead to extra memory allocation.

7) Can we overload the main() method?
➡️ Yes, we can overload the `main()` method. A Java class can have any number of `main()` methods. But to run the Java program, the class should have the `main()` method with the signature "public static void main(String[] args)." If you modify this signature, compilation will be successful, but the program will not run, resulting in a runtime error stating that the main method is not found.

8) Can we override the Java main method?
➡️ No, because `main` is a static method.

2) ||Java coding interview questions ||
hashtag#SDET
1. Reverse a String:
Write a Java program to reverse a given string.
2. Find the Largest Element in an Array:
Find and print the largest element in an array.
3. Check for Palindrome:
Determine if a given string is a palindrome (reads the same backward as forward).
4. Factorial Calculation:
Write a function to calculate the factorial of a number.
5. Fibonacci Series:
Generate the first n numbers in the Fibonacci sequence.
6. Check for Prime Number:
Write a program to check if a given number is prime.
7. String Anagrams:
Determine if two strings are anagrams of each other.

8. Array Sorting:
Implement sorting algorithms like bubble sort, merge sort, or quicksort.

9. Binary Search:
Implement a binary search algorithm to find an element in a sorted array.

10. Duplicate Elements in an Array:
Find and print duplicate elements in an array.

11. Linked List Reversal:
Reverse a singly-linked list.

12. Matrix Operations:
Perform matrix operations like addition, multiplication, or transpose.

13. Implement a Stack:
Create a stack data structure and implement basic operations (push, pop).

14. Implement a Queue:
Create a queue data structure and implement basic operations (enqueue, dequeue).

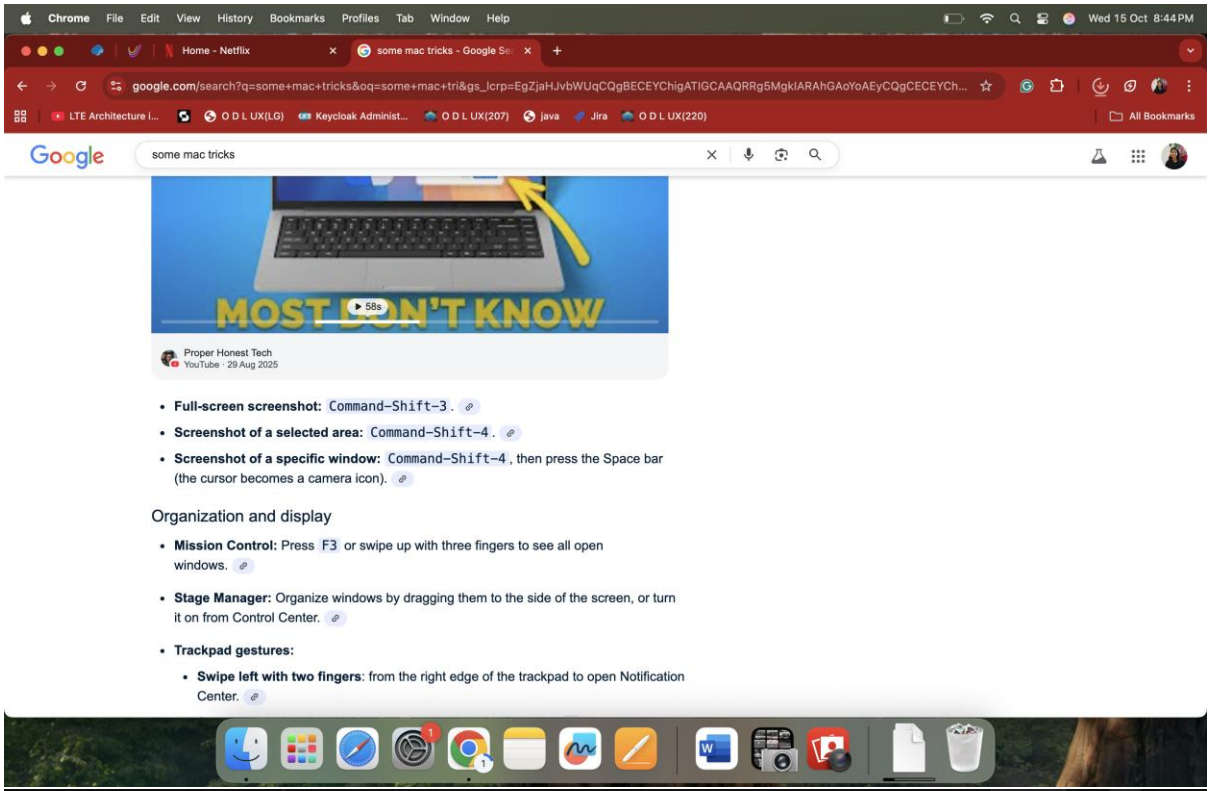
15. Inheritance and Polymorphism:
Implement a class hierarchy with inheritance and demonstrate polymorphism.

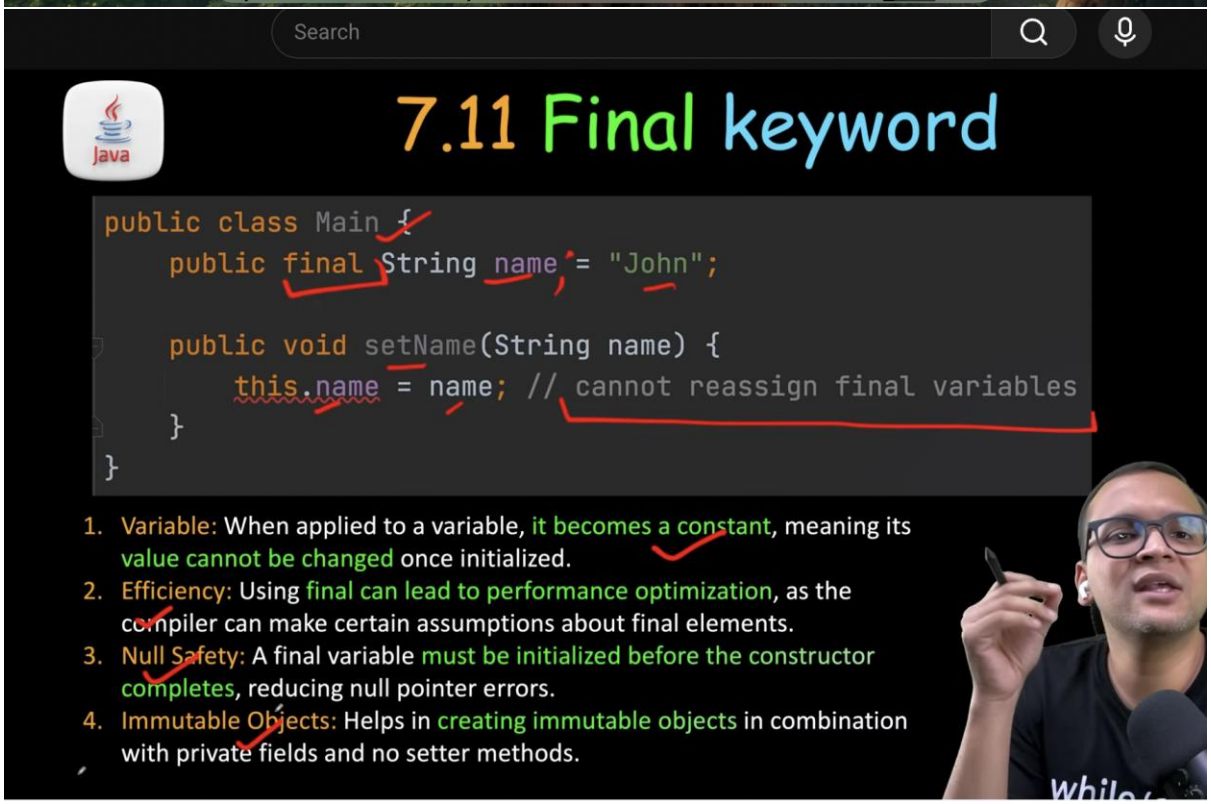
16. Exception Handling:
Write code that demonstrates the use of try-catch blocks to handle exceptions.
17. File I/O:
Read from and write to a file using Java's file I/O capabilities.
18. Multithreading:
Create a simple multithreaded program and demonstrate thread synchronization.
19. Lambda Expressions:
Use lambda expressions to implement functional interfaces.
20. Recursive Algorithms:
Solve a problem using recursion, such as computing the factorial or Fibonacci sequence.
Round 1:
-> Started with your introductions
-> Asked about the last project
-> Reverse the string without using in-built function
-> Explain the concept of polymorphism and its type

-> string concatination :
Input: aaabbbacfwww
Output: a3b3acfw3

3) 1. Implement a function to perform a binary search on a sorted array to find the index of a given target value.
   Input:
   arr = [1, 2, 3, 4, 5, 6, 7, 8, 9]
   target = 5
   Output:
   Index: 4

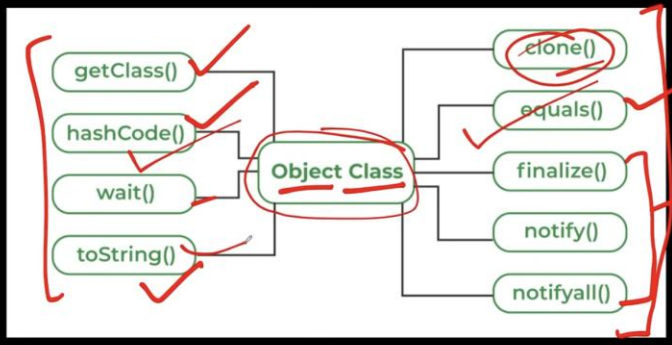
   5. Given an array of integers, write a function to find the maximum product of any two numbers.
   6. Implement a function to detect a cycle in a linked list.
   7. Write a function to find the longest substring without repeating characters.
   8. Describe how you would test a multi-tier application with distributed systems.
   9. How would you write tests for an API with dynamic response data (e.g., timestamps or GUIDs)?
   10. What's your approach to debugging a flaky test in a CI/CD pipeline?
   11. Can you describe a time when you identified a major quality issue in production? How did you handle it?
   12. What is the difference between TCP and UDP? Give real-world examples of their usage and also the three-way handshake process in TCP.
   13. How does DNS resolution work? What is the role of DNS caching?
   14. What is virtual memory, and how does it work?
   15. Write a Linux command to find the top 10 largest files in a directory.





plete Course Part-2 (2024) | 100+ Programming Challenges

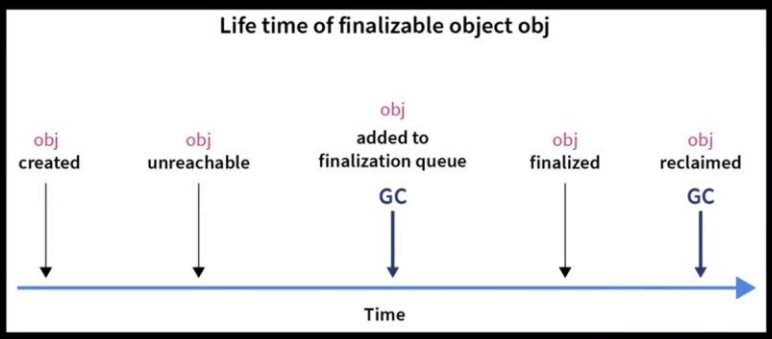
8.8 Object class



1. Root Class: The Object class is the parent class of all classes in Java, forming the top of the class hierarchy.
2. Default Methods: It provides fundamental methods like equals(), hashCode(), toString(), and getClass() that can be overridden by subclasses.
3. String Representation: The toString() method returns a string representation of the object, often overridden for more informative output.



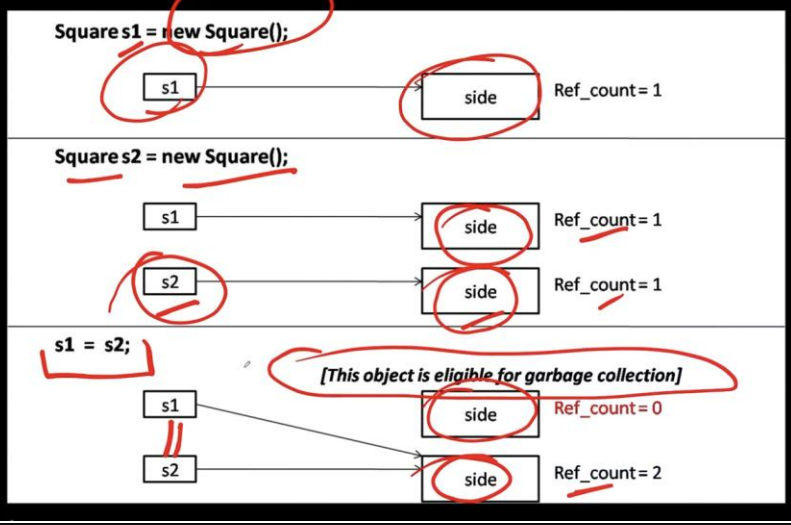
6.10 Garbage Collection & Finalize



1. Finalization: Before an object is garbage collected, the finalize() method may be called, giving the object a chance to clean up resources. However, it's not guaranteed to run, and its usage is generally discouraged.
2. Optimization: Developers can optimize the process indirectly through code practic like setting unnecessary object references to null.
3. System.gc() Call: While System.gc() suggests that the JVM performs garbage collection, it's not a guarantee.

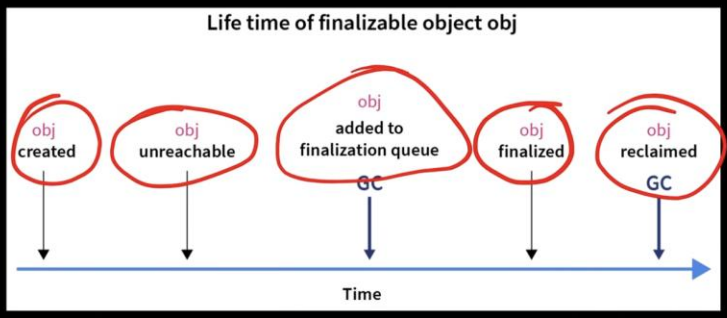


6.10 Garbage Collection & Finalize



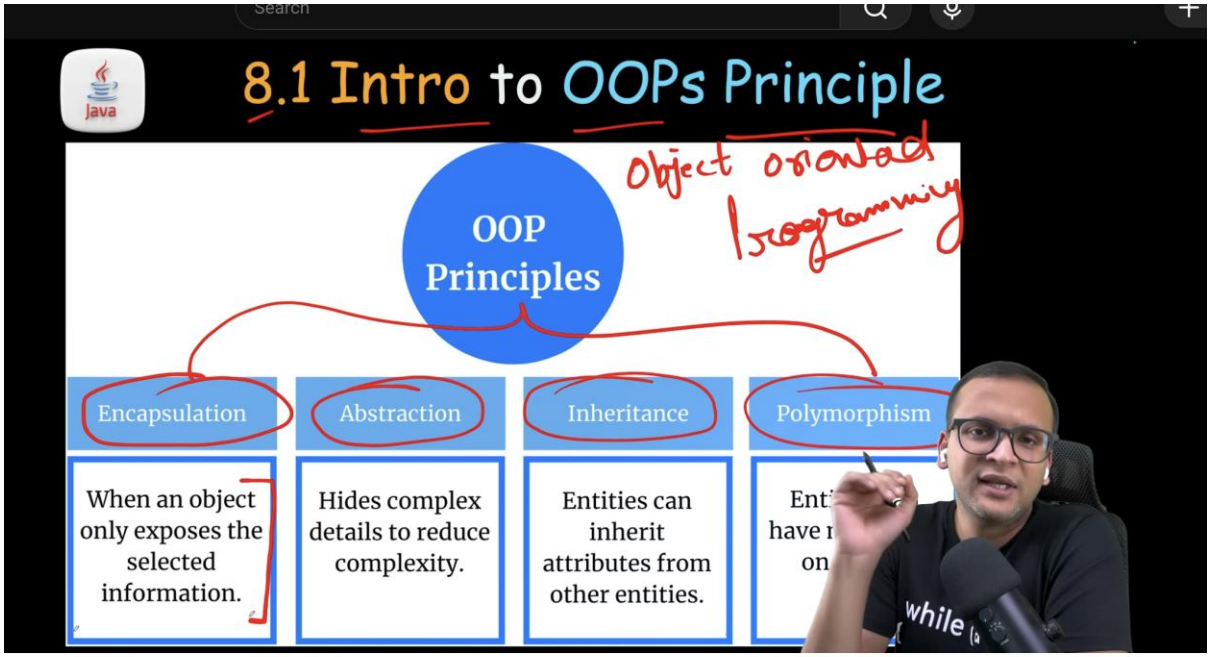


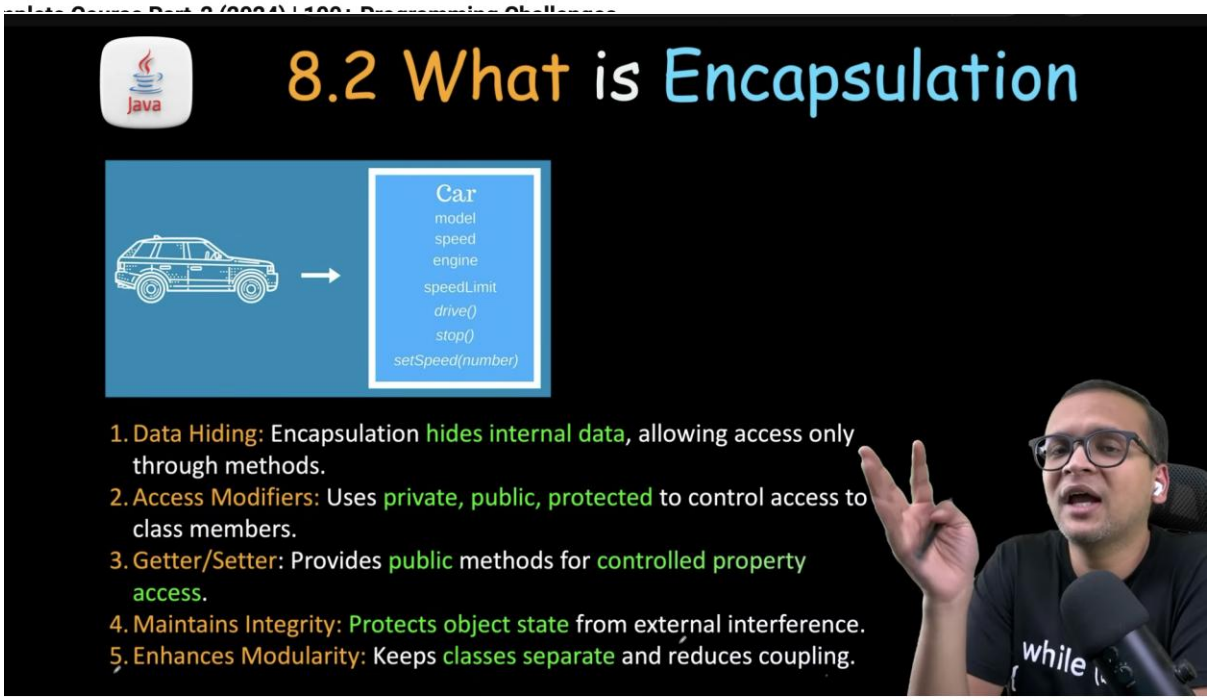
6.10 Garbage Collection & Finalize



1. Finalization: Before an object is garbage collected, the finalize() method may be called, giving the object a chance to clean up resources. However, it's not g ed to run, and its usage is generally discouraged.
2. Optimization: Developers can optimize the process indirectly through code p like setting unnecessary object references to null.
3. System.gc() Call: While System.gc() suggests that the JVM performs garbage collection, it's not a guarantee.







Complete Course Part-2 (2024) | 100+ Programming Challenges

te Coding by Prashant Sir

Subscribe

9K

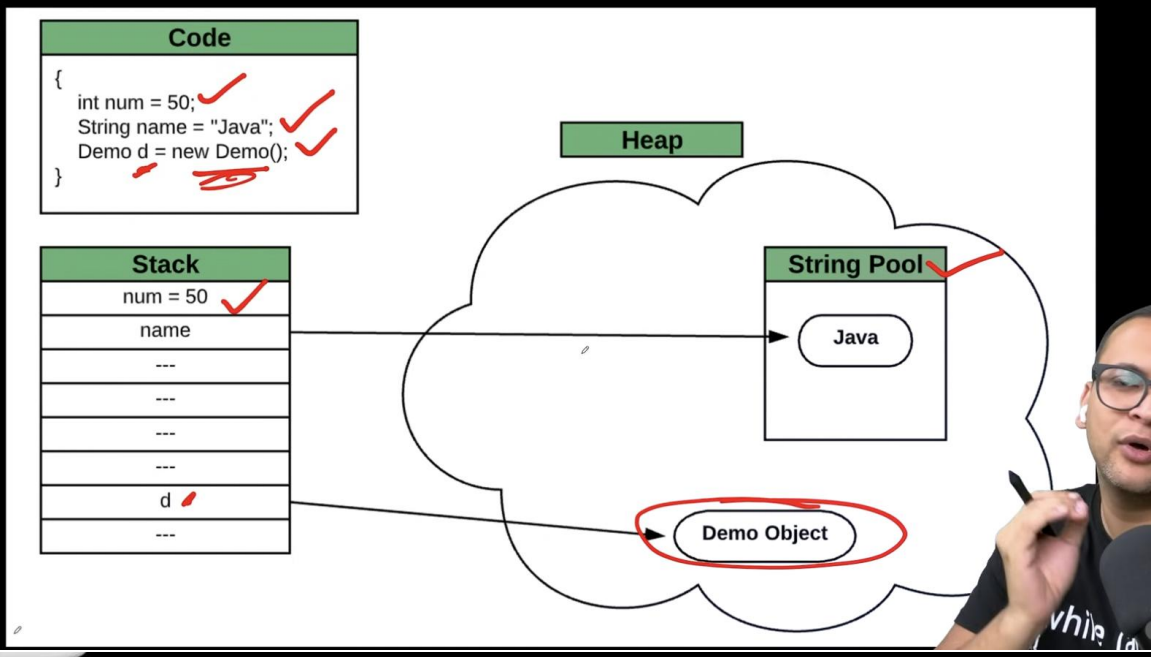
Share

Download

All

From the series

6.7 Stack vs Heap Memory



Instance vs Local Variables

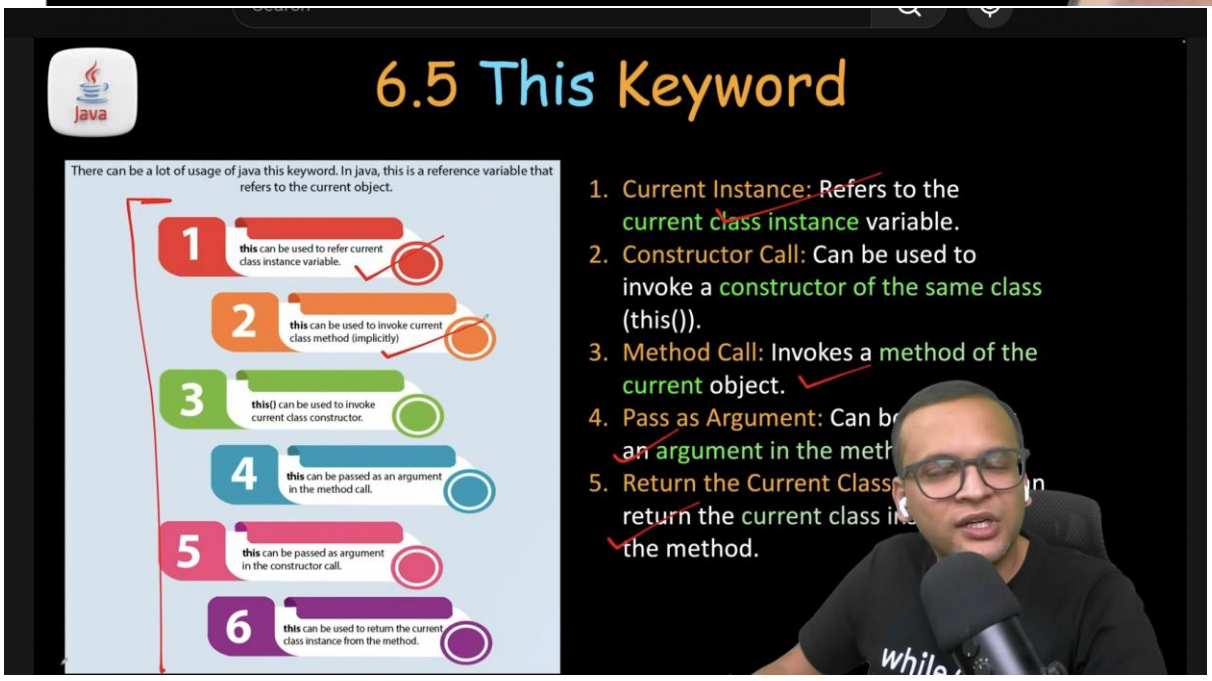
```
class Athlete {
    public String   athleteName;
    public double   athleteSpeed;
    public int      athleteAge;

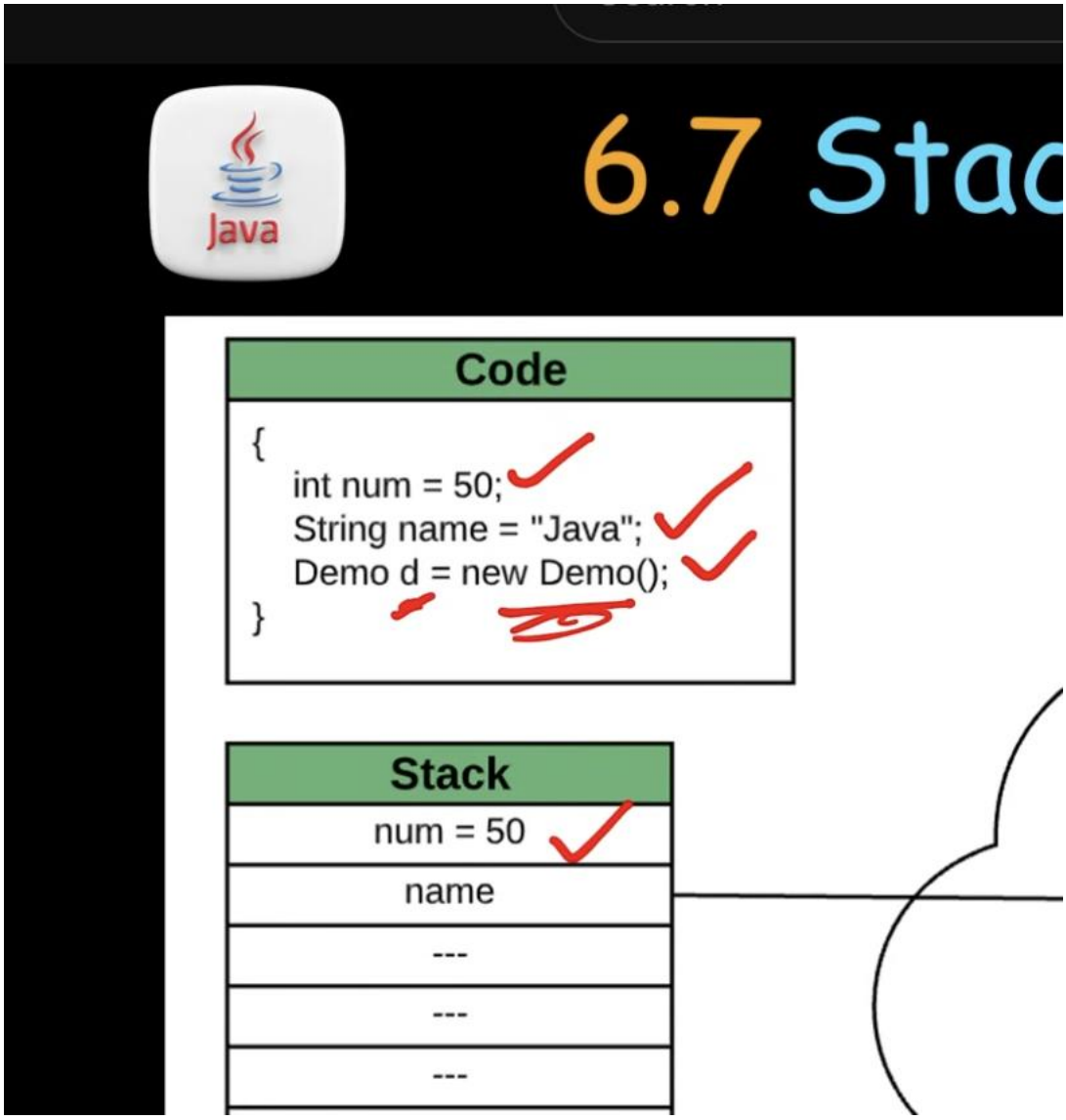
    public Athlete( name. speed, age ){
        this.athleteName = name;
        this.athleteSpeed = speed;
        this.atheleteAge = age;
    }
    public void athleteRun(){
        int speed = 100;
        System.out.println("Athlete runs at"+ speed +"Km/hr");
    }
}
```

Instance Variables

Local Variables

1. Inheritance allows a new class (subclass) to inherit features from an existing class (superclass).
2. Code Reusability: It enables subclasses to use methods and variables of the superclass, reducing code duplication.
3. Access Control: The protected access modifier often used in inheritance to allow subclass acc to superclass members.




6.7 Stac
Code
{
int num = 50;
String name = "Java";
Demo d = new Demo();
}
Stack
num = 50
name

6.10 Garbage Collection & Finc

1. Automatic Process: Garbage collection is managed by the Java Virtual Machine (JVM), running in the background.
2. Object Eligibility: Objects that are no longer reachable, meaning no active references to them, are eligible for garbage collection.
3. No Manual Control: Unlike languages like C++, Java developers cannot explicitly deallocate memory. Garbage collection is automatic and non-deterministic.
4. Generational Collection: Java uses a generational garbage collection strategy, which divides memory into different regions (young, old, and permanent generations) based on object ages.
5. Heap Memory: Garbage collection occurs in the heap memory, where all Java objects reside.
6. Performance Impact: Garbage collection can affect application performance, particularly if it runs frequently or takes a long time to complete.



